



TILERA®

***GETTING STARTED
WITH THE
MULTICORE DEVELOPMENT
ENVIRONMENT***

DOCUMENT RELEASE 1.8
DOC. NO. UG504
NOVEMBER 2014
TILERA CORPORATION

Copyright © 2008-2014 Tilera® Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tilera Corporation: Tilera and the Tilera logo.

The following are trademarks of Tilera Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, iMesh, TileDirect, TILExtreme-Gx, TILExtreme-Gx Duo, TILEmpower, TILEmpower-Gx, TILEmpower-Gx36, TILEmpower-Gx72, TILEncore, TILEncorePro, TILEncore-Gx, TILEncore-Gx9, TILEncore-Gx16, TILEncore-Gx36, TILEncore-Gx72, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx72, TILE-Gx8072, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, TILE-Gx8009, TILE-Gx8016, TILE-Gx8036, TILE-Gx3036, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, TMC (Tilera Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tilera IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tilera products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tilera Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tilera disclaims any express or implied warranty relating to the sale and/or use of Tilera products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN “AS IS” BASIS. Tilera assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number	UG504
Document Release Number	1.8
Date	24 November 2014

Contact Information:

Tilera Corporation
Information info@tilera.com
Web Site http://www.tilera.com

CONTENTS

PREFACE	VII
----------------------	------------

CHAPTER 1 INSTALLING THIS RELEASE

1.1 Understanding the Installation Tarballs	1
1.2 Deciding Which Tarballs to Install	2
1.3 Installing the Primary MDE Software on a Linux x86 Host	2
1.4 Installing the Full MDE Software on a Linux x86 Host	3
1.5 Setting Your Environment Variables	4
1.6 Installing the GCC 4.8 Compiler	4
1.7 Examining Your MDE Installation	4
1.8 Obtaining Source Files	6
1.8.1 Rebuilding RPMs	6
1.8.2 Running the .get Scripts to Get Source Files	6
1.9 Building and Installing the x86 Drivers	6
1.10 Upgrading a TILE-Gx Device to the Latest MDE Image	9
1.11 Exporting the \$TILERA_ROOT/tile Subdirectory	11
1.12 Extracting the Tile Tarball Onto a Persistent Disk	12
1.12.1 Managing RPMs on a Persistent Disk	12
1.13 Installing Packages Required by the MDE IDE	13
1.13.1 Installing Packages for the MDE IDE on RHEL 6.2	13
1.13.2 Installing Packages for the MDE IDE on Other Operating Systems	14
1.14 Configuring SELinux for the x86 Linux Side	14

CHAPTER 2 DOCUMENTATION FOR THE 4.3 RELEASE

2.1 Documents Available Through the MDE Document Index	17
2.1.1 Primary MDE Software Documentation Manuals	17
2.1.2 TILE-Gx Processor Architecture and Hardware Manuals	17
2.1.3 Libraries Documentation	18
2.1.4 Development Tools Documentation	18
2.1.5 Operating System Internal Interfaces	18
2.1.6 Application Notes	18
2.1.7 I/O Device Documentation	18
2.2 Man Pages	19
2.2.1 Section 1 (General Commands) Man Pages	19
2.2.2 Section 2 (System Calls) Man Pages	19

2.2.3 Section 4 (File Formats) Man Pages	19
2.2.4 Section 5 (Miscellaneous) Man Pages	19

CHAPTER 3 EXAMINING THE TILERA MULTICORE DEVELOPMENT ENVIRONMENT

3.1 Overview of the MDE Components	21
3.2 Steps in the Development Process	22
3.3 Hardware and Simulator Development Platforms	22
3.4 Interacting with the Tile System	23
3.4.1 Understanding Tile Boot Processing	24
3.4.2 Understanding Hypervisor Definition Processing	24

CHAPTER 4 RUNNING PROGRAMS USING THE COMMAND-LINE INTERFACE

4.1 Overview	27
4.2 Getting Help on the MDE Commands	27
4.3 Before You Use the Examples	28
4.4 Running the Example Programs on the Hardware Platform	28
4.4.1 Running the Single-Tile Example Program	28
4.4.2 Running the Multi-Tile Example Program	29
4.5 Running the Example Programs on the Simulator	29
4.5.1 Running the Single-Tile Simulator Example Program	29
4.5.2 Running the Multi-Tile Simulator Example Program	30
4.6 Explanation of Output	30
4.7 Where to Go from Here	31

CHAPTER 5 GETTING STARTED WITH THE IDE

5.1 Overview	33
5.2 Bringing Up the Workbench for the First Time	33
5.3 Getting Help on the Tilera IDE	34
5.3.1 Using the Online Help Browser	35
5.3.2 Context-Sensitive Help	36
5.4 About the Eclipse UI and Its Terminology	37
5.4.1 Workspace Directory	37
5.4.2 Project Directory	38
5.4.3 Views	38
5.4.4 Perspectives	39
5.4.5 Projects	39
5.5 Tilera Extensions to Eclipse	39
5.6 Using the Tutorial to Become Familiar with the Graphical User Interface	40
5.6.1 Applications and Programs	40
5.6.2 "Hello World" Application	40
5.6.3 Copying the Examples Directory	41

5.7 Creating a Project	41
5.7.1 Using the Example Application to Create a Project	41
5.7.2 Closing the New Project Wizard	44
5.8 Building the Project	44
5.9 MDE Execution/Monitoring Framework	46
5.10 Running an Application	48
5.11 Running an Application Using a Launch Configuration	48
5.11.1 Creating a Monitor Launch Configuration	48
5.11.2 Closing the Launch Configuration	57
5.11.3 Invoking the Launch Configuration	57
5.12 Continuing the Tutorial	60
CHAPTER 6 FREQUENTLY ASKED QUESTIONS (FAQS)	
6.1 Basic FAQs	61
6.2 FAQs About tile-monitor	61
6.3 FAQs About the IDE	61
6.4 FAQs About the Hypervisor	65
INDEX	67

PREFACE

About This Guide

This guide provides a brief introduction to the Multicore Development Environment™ (MDE) from Tiler® Corporation. The MDE is a standards-based set of tools for multicore programming, enabling developers to take full advantage of the parallel-processing potential of the Tile Processor™ architecture.

This guide describes how to install the MDE software and get up and running quickly.

This guide applies to MDE Release 4.0.

Intended Audience

This guide is intended for use by individuals who are responsible for installing software and by Tile Processor application developers.

Note: This guide assumes that you have already read the *Release Notes for MDE Release 4.1* (UG708).

Changes from the Previous Release

The major changes in this guide from MDE 4.2 are:

- [Chapter 1: Installing This Release](#) provides new procedures for upgrading from MDE Release 4.1x and 4.2.x.

Document Organization

This guide is organized as follows:

- [Chapter 1: Installing This Release](#)
Describes how to install this release.
- [Chapter 2: Documentation for the 4.3 Release](#)
Describes the documentation for the release.
- [Chapter 3: Examining the Tiler Multicore Development Environment](#)
Introduces the MDE.
- [Chapter 5: Getting Started with the IDE](#)
Provides an introduction to the Integrated Development Environment (IDE).
- [Chapter 4: Running Programs Using the Command-Line Interface](#)
Describes how to use the supplied makefiles, compiler, and `tile-monitor` tool to run the examples in the software kit.

- [Chapter 6: Frequently Asked Questions \(FAQs\)](#)

Presents frequently asked questions.

This guide also has an [Index](#).

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tiler tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own [Tilera IDE Online Documentation](#), which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command [Help → Help Contents](#).

- When the online help browser is displayed, select the document [Tilera IDE Online Documentation](#) in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at <http://www.tilera.com>.
- E-mail questions to support@tilera.com.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
[]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
...	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
\$	This is the system prompt for host-side commands.
#	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as <code>root</code> .
<code>command</code>	A command name appears in the Courier New font
<code>computer output</code>	Computer output appears in the Courier New font.
<i>filename</i>	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
<i>new term</i>	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
TILERA_ROOT	TILERA_ROOT is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tiler tools and examples. We recommend that you use the <code>tile-env</code> script described in <i>Getting Started with the Multicore Development Environment</i> (UG504) to set it correctly.

CHAPTER 1 INSTALLING THIS RELEASE

This chapter describes how to install this release:

- [Section 1.1 Understanding the Installation Tarballs](#)
- [Section 1.2 Deciding Which Tarballs to Install](#)
- [Section 1.3 Installing the Primary MDE Software on a Linux x86 Host](#)
- [Section 1.4 Installing the Full MDE Software on a Linux x86 Host](#)
- [Section 1.5 Setting Your Environment Variables](#)
- [Section 1.6 Installing the GCC 4.8 Compiler](#)
- [Section 1.7 Examining Your MDE Installation](#)
- [Section 1.8 Obtaining Source Files](#)
- [Section 1.9 Building and Installing the x86 Drivers](#)
- [Section 1.10 Upgrading a TILE-Gx Device to the Latest MDE Image](#)
- [Section 1.11 Exporting the \\$TILERA_ROOT/tile Subdirectory](#)
- [Section 1.12 Extracting the Tile Tarball Onto a Persistent Disk](#)
- [Section 1.12.1 Managing RPMs on a Persistent Disk](#)
- [Section 1.13 Installing Packages Required by the MDE IDE](#)
- [Section 1.14 Configuring SELinux for the x86 Linux Side](#)

Note: If you are upgrading from an earlier Multicore Development Environment™ (MDE) release, you must rebuild any of the applications that you intend to use with this latest MDE release.

1.1 Understanding the Installation Tarballs

The MDE 4.3 release includes the following tarballs:

- **Primary tarball:** `TileramDE-version_arch.tar`
This tarball contains most of the files for the MDE environment, including x86 host-based tools, user-space utilities, libraries, header files, and binary run-time components. It also contains the *basic* Tile tarball.
- **Basic Tile tarball:** `tile.tar.xz`
This tarball is contained within the Primary tarball. It includes native Tile files and tools such as the `tile-gcc/g++` compiler and Linux, and about 160 of the most useful RPMs, and appears in the `$TILERA_ROOT` directory. The `unpack` command extracts this tarball into the `$TILERA_ROOT/tile` directory.
- **Full Tile tarball:** `TileramDE-version_arch_tile_full.tar.xz`

This optional tarball is a more complete version of the *basic* Tile tarball. It includes all the native Tile files and tools and RPMs in the *basic* Tile tarball and about 4,000 additional RPMs, which are available in the RPMS tarball.

- RPM tarball: `TileraMDE-version_arch_rpms.tar`

This optional tarball contains all of the RPMs that populate the *full* Tile tarball.

- GCC 4.8 tarball: `TileraMDE-version_arch_gcc48.tar.bz2`

This optional tarball contains cross-compiler binaries and native tools for using the GCC 4.8 compiler to augment the default GCC 4.4 compiler.

- Primary Debug tarball: `TileraMDE-version_arch_dbg.tar.bz2`

This tarball contains debug symbols for the binaries in the Primary Tile tarball.

- RPM Debug tarball: `TileraMDE-version_arch_rpms_dbg.tar`

This tarball contains This tarball contains the `debuginfo` RPM packages. Use these to debug binaries from the RPMs in the RPMS tarball.

1.2 Deciding Which Tarballs to Install

Every site using MDE 4.3 must install the Primary tarball, `TileraMDE-version_arch.tar`, and either the *basic* Tile tarball or the *full* Tile tarball.

Decisions you need to make are:

- Whether to install the *basic* Tile tarball or the *full* Tile tarball.
- If you decide to install the *basic* Tile tarball, whether or not to install additional RPMs and which ones to install.
- If you want to use the gcc 4.8 compiler, install the `TileraMDE-version_arch.gcc48.tar.bz2` tarball. For details on installing the gcc 4.8 tools, see [Section 1.6 “Installing the GCC 4.8 Compiler” on page 4](#).

Note: You can install either the *basic* or *full* tile tarball, not both. Once you run the `unpack` command, you cannot run it again without deleting the entire contents of the `tilegx` sub-directory structure.

1.3 Installing the Primary MDE Software on a Linux x86 Host

Follow the steps below for any x86 machine on which you need the basic version of the MDE installed.

Replace *version* with the version number of the MDE and *arch* with the architecture of your chip (`tilegx`) in each of the steps below.

1. Create an installation directory:

```
$ mkdir -p /opt/tilera
```

Note: We suggest `/opt/tilera` as an installation location for the MDE, so that it is visible to all users on your system.

2. Go to your installation directory:

```
$ cd /opt/tilera
```

3. Acquire the Primary tarball and copy it into `/opt/tilera`.

4. If necessary, decrypt the tarball:

```
$ gpg TileramDE-version_arch.tar.gpg
```

 When prompted, enter the “passphrase” (password) supplied by Tileram.
5. Extract the Primary tarball:

```
$ tar -xf TileramDE-version_arch.tar
```

 This creates the file hierarchy:

```
/opt/tilera/TileramDE-version/arch
```
6. Install the “basic” Tile tarball, using the `unpack` script:

```
$ ./TileramDE-version/arch/unpack
```

 This creates the file hierarchy:

```
/opt/tilera/TileramDE-version/arch/tile
```

1.4 Installing the Full MDE Software on a Linux x86 Host

If you need to install the entire set of RPMs for development, follow the steps below for any x86 machine on which you need the full version of the MDE installed.

Note: You can install either the *basic* or *full* tile tarball, not both. Once you run the `unpack` command, you cannot run it again without deleting the entire contents of the `tilegx` sub-directory structure.

In this and all the following steps, replace *version* with the version number of the MDE and *arch* with the architecture of your chip (`tilegx`).

1. Create an installation directory:

```
$ mkdir -p /opt/tilera
```

Note: We suggest `/opt/tilera` as an installation location for the MDE, so that it is visible to all users on your system.
2. Go to your installation directory:

```
$ cd /opt/tilera
```
3. Acquire the Primary tarball and copy it into `/opt/tilera`.
4. If necessary, decrypt the tarball:

```
$ gpg TileramDE-version_arch.tar.gpg
```

 When prompted, enter the “passphrase” (password) supplied by Tileram.
5. Extract the Primary tarball:

```
$ tar -xf TileramDE-version_arch.tar
```

 This creates the file hierarchy:

```
/opt/tilera/TileramDE-version/arch
```
6. Acquire the *full* Tile tarball and copy it into `/opt/tilera/`
7. If necessary, decrypt the tarball:

```
$ gpg TileramDE-version_arch_tile_full.tar.xz.gpg
```

 When prompted, enter the “passphrase” (password) supplied by Tileram.

8. Install the *full* Tile tarball:

```
$ ./TileramDE-version/arch/unpack \
  TileramDE-version_arch_tile_full.tar.xz
```

This creates the file hierarchy:

```
/opt/tilera/TileramDE-version/arch/tile
```

1.5 Setting Your Environment Variables

The MDE includes a script named `tile-env` that emits shell commands that set the `TILERAMDE_ROOT` environment variable to the root of the installation and adds `$TILERAMDE_ROOT/bin` to the `PATH` environment variable so that the commands and utilities of the MDE installation are more easily accessible.

Each user who wants to use the software should add the following (note the use of back-ticks, not single quotes) to their `$HOME/.profile` or `$HOME/.cshrc` file:

```
eval `./opt/tilera/TileramDE-version/arch/bin/tile-env`
```

You can also enter this command interactively in any shell.

1.6 Installing the GCC 4.8 Compiler

If you need to use the `gcc` 4.8.2 compiler tools, follow the steps below for any x86 machine on which you need the full version of the MDE installed.

In this and all the following steps, replace *version* with the version number of the MDE and *arch* with the architecture of your chip (`tilegx`).

1. Go to your installation directory:

```
$ cd /opt/tilera
```

2. Acquire the GCC48 tarball and copy it into `/opt/tilera`.

3. If necessary, decrypt the tarball:

```
$ gpg TileramDE-version_arch_gcc48.tar.bz2.gpg
```

When prompted, enter the “passphrase” (password) supplied by Tileram.

4. Extract the GCC48 tarball:

```
$ tar -xjf TileramDE-version_arch_gcc48.tar.bz2
```

This provides you with GCC48 cross-compiler binaries and support files for the x86 host places the file `tile_gcc48.tar.xz` into the `$TILERAMDE_ROOT` directory.

5. use `cat`, `ssh`, `xzdec` and `tar` to install the GCC47 tarball on the TILE-Gx device:

```
$ cat /opt/tilera/TileramDE-version/arch/tile_gcc48.tar.xz \
| ssh -x root@addr 'xzdec | tar -C / -xpf -'
```

1.7 Examining Your MDE Installation

After you extract the primary MDE tarball the MDE root directory has several subdirectories. If you have set your `TILERAMDE_ROOT` environment variable properly, you can `cd` to `$TILERAMDE_ROOT` and see the subdirectories:

```
bin  doc  etc  examples  include  lib  man  share  src  tile  tile.tar.
xz
```

The following list reflects the structure of the MDE software and describes the contents of each directory:

- `bin/`: Development tools.
- `etc/`: Miscellaneous internal files used by the MDE software.
- `include/`: Files useful for host-side programs, such as host-side PCI support.
- `lib/`: Various libraries and support files used by the development tools.
- `man/`: Man pages for development tools.
- `share/`: Oprofile support
- `tile/`: Tile Processor architecture-specific files. Some of the files are:
 - `bin/`, `sbin/`, `usr/bin/`, `usr/sbin/`: Native Linux commands.
 - `boot/`: OS components, including the standard `vmlinux` image, `initramfs.cpio.xz`, and the `sromboot.bin` booter file.
 - `etc/`: Default Linux configuration files.
 - `etc/hvc/`: Hypervisor configuration files.
 - `lib/`, `usr/lib/`: Libraries for the standard C library and other APIs.
 - `usr/include/`: Header files for the standard C library and other APIs.
 - `usr/share/`: Support files for various packages plus documentation
 - `var/`: Support files for various packages.
 - `usr/tilera/doc/`: Online documentation.
 - `index.html` (Tilera MDE Document Index, with links to all the MDE documentation, including the guides and manuals, application notes, technical notes, tools documentation, and libraries documentation)
 - `html/`: HTML documentation, including Tile tool descriptions
 - `pdf/`: PDF documents
 - `usr/tilera/examples/`: Examples to help you use the MDE.
 - `dataplane/`: Zero Overhead Linux™ (ZOL) examples.
 - `getting_started/`: The “Hello World” tutorial example in the [Tilera IDE Online Documentation](#) and examples from this guide.
 - `mica/`: Examples of how to use MiCA.
 - `mpipe/`: Examples of how to use mPIPE.
 - `simio/`: Examples for simulator I/O.
 - `tilegx/`: Examples using various features of the TILE-Gx processor.
 - `tmc/`: Example TMC™ (Tilera Multicore Components) programs describing many useful primitives for building parallel programs.
 - `trio/`: Examples of how to use TRIO.
 - `usr/tilera/src/`: Optional source files.
 - `.get` scripts.
 - `sys/`: Sources for operating system components.
 - `tools/`: Sources for development tools and run-time library components.

The MDE installation process establishes three symlinks, which point to files that actually reside in `$TILERA_ROOT/tile/usr/tilera/`. The symlinks are provided for convenience:

- `doc/`
- `examples/`
- `src/`

1.8 Obtaining Source Files

You may obtain further source files using any of the following methods.

1.8.1 Rebuilding RPMs

Most of the Tile packages in the Tile tarballs are built from SRPMs (source RPMs). A source RPM contains the source code corresponding to that RPM or the scripts of a non-compiled software package. By default, the RPMs are created in `/root/rpmbuild/RPMS/`.

If you need to rebuild one of these packages (for example, from a more recent version of an SRPM), you can use the `rpmbuild-tilera` script on the Tile side. This script invokes `rpmbuild` and automatically applies Tilera-specific patches:

```
# rpmbuild-tilera --rebuild SRPM
```

Then you can run `yum` or `rpm -i` to install the package.

1.8.2 Running the .get Scripts to Get Source Files

In the `$TILERA_ROOT/src` directory, there are several files with a `.get` suffix. Some examples are:

- `linux-version.get`
- `gcc-version.get`
- `glibc-version.get`

To obtain the sources for the corresponding packages, you can run the individual `.get` scripts, which acquire the official sources, unpack them, apply Tilera-specific patches to them, and place them in the current directory.

For example, if you want to place the sources in a directory called `$HOME`, `cd` to that directory and then do the following:

```
$ $TILERA_ROOT/src/linux-version.get
```

After you run this script, the Linux source files are available in `$HOME/linux-version`.

For more information, see the `README` file in the `$TILERA_ROOT/src` directory and Section 3.1 “Building Linux from Source” in *Multicore Development Environment System Programmer’s Guide* (UG509).

1.9 Building and Installing the x86 Drivers

To build and install the x86 USB and/or PCI host drivers on your host machine, perform the following steps.

Note: If you use a TILEncore-Gx Intelligent Application Adapter, you should re-compile USB and/or PCI host drivers.

1. Run the `$TILERA_ROOT/lib/modules/tileusb-compile` script to build the x86 USB driver and the `$TILERA_ROOT/lib/modules/tilepci-compile` script to build the x86 PCI driver for your OS version.

Use the `$TILERA_ROOT/bin/config-tileusb` script to assign names to appliances that you need to access over a USB connection.

Tilera includes pre-built drivers for Red Hat Enterprise Linux 5, CentOS 5, Red Hat Enterprise Linux 6, and CentOS 6 with the following kernel versions:

- 2.6.18-128.el5-x86_64
- 2.6.18-164.11.1.el5-x86_64
- 2.6.18-164.el5-x86_64
- 2.6.18-194.el5-x86_64
- 2.6.18-238.el5-x86_64
- 2.6.18-274.el5-x86_64
- 2.6.18-308.el5-x86_64
- 2.6.18-53.el5-x86_64
- 2.6.18-8.el5-x86_64
- 2.6.18-92.el5-x86_64
- 2.6.32-131.0.15.el6.x86_64
- 2.6.32-220.el6.x86_64
- 2.6.32-279.5.2.el6.x86_64
- 2.6.32-71.el6.x86_64

2. Login as root and run the driver installation script:

```
# $TILERA_ROOT/bin/install-drivers [serial0 ...]
```

The argument(s) include the Linux device name(s) for the serial device to be used for each of the cards or appliances that you have installed.

The `install-drivers` script installs the x86 device drivers. The device driver itself populates all of the `/dev/tile*` directories. For each argument, this script updates the `/etc/tile*.conf` files, which are associated with the card or appliance. If you supply the `none` argument for any card or appliance, this script removes any configuration for serial devices from the `tile*.conf` associated with that card or appliance.

The `$TILERA_ROOT/bin/install-drivers` script uses the `$TILERA_ROOT/lib/modules/tileusb-install` and `$TILERA_ROOT/lib/modules/tilepci-install` scripts.

This serial device path is used by `tile-monitor` and `tile-console` to interact with the serial cable attached to the TILEncore-Gx adapter. You can use the TILEncore-Gx adapter without installing a serial cable, although you will not see console messages displayed while the TILEncore-Gx adapter is booting nor use the console connection for debugging.

- Configuring TILEncore-Gx adapter to use a UART Console Cable

If you have only one TILEncore-Gx Intelligent Application Adapter installed and its UART connector (serial cable) is connected to `/dev/ttyS0`, run:

```
# $TILERA_ROOT/bin/install-drivers /dev/ttyS0
```

If you have only one TILEncore-Gx Intelligent Application Adapter installed and are using no serial cable, run:

```
# $TILER_ROOT/bin/install-drivers
```

If your system includes three TILEncore-Gx Intelligent Application Adapters and devices and we are supplying their three serial port definitions, run:

```
# $TILER_ROOT/bin/install-drivers /dev/ttyS0 none /dev/ttyUSB3
```

- Configuring TILEncore-Gx Adapter to Use a USB Console Cable

If you have only one TILEncore-Gx Intelligent Application Adapter installed and are using the USB connector cable to access the adapter over the USB console, run:

```
# $TILER_ROOT/bin/install-drivers /dev/tileusb0/console
```

Note: Run the `install-drivers` script whenever the serial devices associated with your adapters or appliances change, including whenever you install a new adapters and/or you change the serial ports to which the serial cables from the adapters are connected.

3. Connect to the console of the TILEncore-Gx adapter using the `tile-console` utility provided in the MDE. If you have only one TILEncore-Gx Intelligent Application Adapter installed in the host machine, use the command:

```
$ tile-console --dev gxpci0
```

This `tile-console` command connects you to the default console device that you established using the `install-drivers` command. To see the default setting for TILEncore-Gx adapter console device, examine the contents of the device file using the command:

```
$ cat /etc/tilegxpci0.conf
```

If you configured the TILEncore-Gx adapter to run over a UART console cable, the device file contents are:

```
serial= /dev/ttyS0
```

If you configured the TILEncore-Gx adapter to run over a USB console cable, the device file contents are:

```
serial= /dev/tileusb0/console
```

4. To verify the MDE installation on the Linux x86 host, use this command on the host in which the TILEncore-Gx Intelligent Application Adapter is installed:

```
$ tile-monitor --dev device_specification -- cat /proc/version
```

By default, the `device_specification` is `gxpci0`. The command outputs simple version information, connects to the default hardware, boots it with a default boot configuration, and displays the booted software version. This validates that the MDE software can connect to and initialize the TILE-Gx hardware.

1.10 Upgrading a TILE-Gx Device to the Latest MDE Image

This procedure describes how to update the MDE software on a platform that contains a Tiler TILE-Gx chip and boots to the TILE filesystem on persistent (disk-based) storage.

This procedure applies to the following platforms:

- TILEmpower-Gx36™ (including the TILEmpower-Gx36 MP and TILEmpower-Gx36 FR)
- TILEmpower-Gx72™ (including the TILEmpower-Gx72 MP and TILEmpower-Gx72 FR)
- TILExtreme-Gx36™

You might have to perform additional steps (perhaps partitioning and formatting drives) for platforms running earlier releases. Please contact support@tilera.com if you need more help.

Note: If you need to upgrade the MDE installation on a TILExtreme-Gx36 Duo platform, **do not use this procedure**. Instead, see the directions that the MDE software installation process creates at:

```
$TILER_A_ROOT/doc/html/duo_software_install_iscsi.html
```

Tilera ships a TILE-Gx-based platform with three partitions (early shipments vary):

/dev/sda1	20 GB	ext4 FS
/dev/sda2	20 GB	swap partition
/dev/sda3	90 GB	ext4 FS

Note: The third partition's size is based on the size of the SSD.

This procedure assumes that you:

- Need to upgrade the platform from the platform itself (i.e. within the Tilera native environment and not using cross-development tools such as `tile-monitor`).
- Mount `sda3` as `/home` and use it for user data that persists between MDE updates. We assume `sda1` holds mostly the contents of the Tilera distribution, and that most changes to this section are minimal, or will otherwise be tracked and repeatable via procedures and scripts.
- Upload packages from the new MDE release, and assume the existence of a directory (on `sda3`) called `/home/tilera/MDEs/`. You must have the connectivity and permissions to use the `scp` command to copy files into that directory.
- Boot the platform from SPI-ROM. One slot contains the `recovery.bootrom` file. This slot is locked and has the lower generation number (usually with a comment indicating that it contains the recovery bootrom). You may verify this by reading the output of this command:

```
# sbim
```

The bootrom slot with the lower *gen* number should have a comment identifying it as a *recovery* image. If needed, copy it to the current directory and burn it in now, using this command:

```
# sbim -i /usr/lib/boot/recovery.bootrom -v \
> -c "recovery.bootrom MDE current_version_arch"
```

Consult the `sbim` manual page for more information and to learn how to promote the gen numbers and lock slots.

In this procedure, we assume that:

- The TILE-Gx-based platform is booted up.

- You are logged on as `root` on the platform's console (you need early access during the boot).
- The `#` is the prompt for the TILE-Gx-based platform.

Instructions:

1. Boot the TILEmpower-Gx platform with the recovery image, shutting down the platform properly before rebooting:

```
# reboot
```

To boot the image with the low generation number, which is the alternate boot and the recovery bootrom, you have to press the `a` key on the console when prompted during the boot.

Note: With early MDE releases you might need to power-cycle the platform if the previous boot was not from SPI-ROM.

2. Mount the partitions and perform backups, using these commands:

```
# mount /dev/sda1 /mnt
# mount /dev/sda3 /home
```

- a. Back up the filesystem table and time zone:

```
# cp /mnt/etc/fstab /home/tilera/
# cp /mnt/etc/localtime /home/tilera/
```

- b. Back up any other files you might need on `/dev/sda1`.

3. Create a download directory and get the full Tile filesystem tarball, using these commands:

```
# cd /home/tilera/MDEs
# mkdir MDE-version_arch
# cd MDE-version_arch
# scp myname@myhost:/Tilera/download/path/TileraMDE-
version_arch_tile_full.tar.xz .
```

4. Rebuild the filesystem on `sda1` and install new contents:

```
# umount /mnt
# mkfs.ext4 /dev/sda1 -L root
# mount /dev/sda1 /mnt
# xzdec TileraMDE-version_arch_tile_full.tar.xz | tar -C /mnt -xpf -
```

This process should take less than 25 minutes to complete.

Note: It is important that the `root` partition is labeled correctly since MDE Release 4.3.0 or later relies on partition labels to identify the `root` partition. Use the `-L` parameter with `mkfs.ext4` as shown above. You can check partition labels using the `blkid` command. An example of the output is shown below where the label for the partition is indicated as `root`:

```
/dev/sda1: LABEL="root" UUID="93d2fe28-ca02-4c6d-a6a3-
15d803441b06" TYPE="ext4"
```

5. Restore files.

- a. Restore the previous filesystem table (but keep a copy of the default table for safety), time, and other backups:

```
# cd /mnt/etc
# mv fstab fstab.orig
# cp /home/tilera/fstab /home/tilera/localtime .
```

The only expected difference in `fstab` is the first three lines must be un-commented.

- b. Restore any files you have backed up.

6. Update the SPI-ROM, (again, this assumes that the recovery bootrom slot is locked):

```
# sbim -i /mnt/usr/lib/boot/tilempower.bootrom -v -c "MDE-version
tilempower.bootrom"
```

Note: You can substitute the `tilextreme.bootrom` or a custom bootrom as appropriate.

To verify your actions, you can examine the output of:

```
# sbim
```

7. Reboot the platform.

```
# reboot
```

Note: On the TILExtreme-Gx36 and TILExtreme-Gx36 Duo platforms, you must reboot all nodes at the same time. Instead of using the `reboot` command, you should issue a `poweroff` command on each node and then power cycle the whole platform.

Note: If the platform displays the repeating message: Waiting for partition LABEL=root to become available, it indicates that the root partition is not labeled correctly.

If the root partition is labeled incorrectly, you need to do the following:

- a. Reboot into the recovery bootrom by power-cycling the platform and then typing a at the prompt.
- b. Use the `e2label` command to label the root filesystem properly:

```
# e2label /dev/sd1 root
```

- c. Reboot.

```
# root
```

1.11 Exporting the \$TILERA_ROOT/tile Subdirectory

You can gain access to the files from the Tile tarball, which were extracted into the `$TILERA_ROOT/tile/` directory in [Section 1.3 “Installing the Primary MDE Software on a Linux x86 Host” on page 2](#), if you do not have a persistent disk.

The easiest way to do this is to use `tile-monitor --root`. This command mounts and uploads the interesting subdirectories of the `$TILERA_ROOT/tile/` directory.

You can also use either of the following techniques, but you must know exactly which files you need:

- Upload specific files from the `$TILERA_ROOT/tile/` directory, using a technique such as `scp` or `tile-monitor --upload-tile`.

- Create a custom Linux image whose in-RAM filesystem (initramfs) contains some of the files in the `$TILERA_ROOT/tile/` directory. See the “Controlling the Linux Initramfs Boot Process” section in the “Linux Interfaces and Customization” chapter in the *Multicore Development Environment System Programmer’s Guide* (UG509).

1.12 Extracting the Tile Tarball Onto a Persistent Disk

The technique described below extracts all the files in the “basic” or “full” Tile tarball onto a persistent disk, using the `tile-monitor --root` command, as described in [Section 1.11 “Exporting the \\$TILERA_ROOT/tile Subdirectory” on page 11](#).

Your persistent disk must be 2GB.

1. Boot the TILE system, where `addr` is the address of the TILE system.

Assuming that the disk partition has already been created, use this command:

```
$ tile-monitor --net addr --root --quit
```

If you must create the persistent disk partition, use a command like this:

```
$ tile-monitor --net addr --root \
  --run - fdisk /dev/sda - \
  --run - mkfs.ext4 /dev/sda1 - \
  --quit
```

2. Use `cat`, `ssh`, `xzdec` and `tar` to install the Tile tarball:

```
$ cat /opt/tilera/TileramDE-version_arch tile.tar.xz \
  | ssh -x root@addr 'mount /dev/sda1 /mnt; xzdec | tar -C /mnt -xpf -'
```

Note: The `xzdec` command is available in `$TILERA_ROOT/bin`

3. Run the `vi` editor as root to modify the `/mnt/etc/fstab` file, using the command:

```
chroot vi /mnt/etc/fstab
```

4. Modify the file `/mnt/etc/fstab` to create (or un-comment if it is already available) the following line:

```
/dev/sda1          /          ext4      defaults      1 1
```

Note: This `fstab` entry assumes that the `/dev/sda1` drive is an `ext4` type file system. If the file system type is different (as reported by the output of `/etc/mounts`), then the line should be modified accordingly.

5. To burn a bootrom into the SROM which will boot from this disk partition, use this command:

```
$ tile-monitor --net addr --rootfs /dev/sda1 -- mount
```

During this reboot (and all future reboots), the system will use the hypervisor, the `hvconfig` file, and `vmlinux` from the SROM. Linux then uses the files on the disk partition as the standard file system.

1.12.1 Managing RPMs on a Persistent Disk

If you extract the “full” Tile tarball, you have all the Tileram RPMs. If not, install the RPMS tarball into the `$TILERA_ROOT` directory.

1.12.1.1 Installing RPMs on a Persistent Disk

To install an RPM on a persistent disk, extract the entire RPMS tarball and use the `yum` utility to install individual packages.

1. In the `$TILERERA_ROOT` directory, install the entire RPMS tarball:

```
$ tar -xf TileramDE-version_arch_rpms.tar
```
2. Use the `yum` command to list all installed packages:

```
$ yum list installed
```

You could also use the `rpm` command to list the installed packages:

```
$ rpm -qa
```
3. Use the `yum` command to install the RPM packages that you want:

```
$ yum install package_file
```

You could also use the `rpm` command to install packages:

```
$ rpm -i package_file
```

1.12.1.2 Removing RPMs from a Persistent Disk

To remove an RPM from a persistent disk, use `yum remove` or `rpm -e`.

```
$ yum remove package_file
```

You could also use the `rpm` command to install packages:

```
$ rpm -e package_file
```

Note: The `yum` utility displays information about other RPM packages that are dependent upon the package that you want to remove. The `rpm -e` command displays similar information.

1.13 Installing Packages Required by the MDE IDE

To run the Multicore Development Environment IDE on a Linux x86 host machine, you must ensure that your host OS supports GTK+ and 32-bit libraries.

Keep the following requirements in mind:

- The `tile-eclipse` command requires GTK+ (a.k.a. GTK2).
- The `tile-eclipse` IDE is a 32-bit application, and requires the 32-bit versions of packages, which may not be installed by default on some 64-bit Linux installations.

Note: These 32-bit packages are only required on hosts on which you run the IDE; you do not need them for other MDE tools (the compiler, debugger, etc.).

1.13.1 Installing Packages for the MDE IDE on RHEL 6.2

To support the Multicore Development Environment IDE on a Linux x86 host machine running RHEL 6.2, do the following:

1. Modify your YUM configuration file (`/etc/yum.conf`) to have these settings:

```
group_package_types = mandatory default optional
multilib_policy=all
```

2. Use YUM to install the following packages:

```
yum install glibc
yum install gtk2-2.18.9-4.el6.i686
yum install gtk+extra-2.1.1-13.el6.i686
yum install gtk+extra-devel-2.1.1-13.el6.i686
yum install glpk-4.40-1.1.el6.i686
yum install glpk-*
yum install PackageKit*
yum install libcanberra*
```

1.13.2 Installing Packages for the MDE IDE on Other Operating Systems

Depending upon your particular host OS, and within the scope of whatever Linux package management system that OS uses, you must somehow install support for:

- 32-bit binaries on a 64-bit machine
- `glibc`
- GTK+, with a version of at least 2.18.x
- Gnu Linear Programming Kit (`glpk`), version 4.40 or greater
- `libcanberra`

Note: The RHEL package management system performs automatic package dependency checking for you. On other operations systems, you may need to perform manual package dependency checking.

1.14 Configuring SELinux for the x86 Linux Side

Note: This section applies to sites that have a Linux distribution on the x86 Linux side that includes SELinux.

Many Linux distributions include SELinux (Security Enhanced Linux). SELinux is a Linux extension that supports the definition and enforcement of access control policies, at a finer level of control than the normal UNIX file permissions (read, write, execute) allow.

Linux distributions that include SELinux also include a default security policy. While this is usually fine for most normal usage, this default security policy might be too strict in some cases. Also, if your site has customized the SELinux security policy, it might be more strict than the default security policy. SELinux might disallow the running of executables and/or libraries that conflict with the SELinux policy in force.

You might experience the following behavior on the host side:

- A binary application you attempt to run fails with an SELinux warning something like this:
Error loading `libcg.so`: cannot restore segment prot after reloc: Permission denied.
- An application or tool fails to run with an unexpected error; for example, if you invoke `tile-eclipse` in an SELinux environment with a policy that is too strict, it might fail to start with a JVM Terminated, Error=-1 message.

In this latter case, SELinux itself could report an error like:

```
SELinux is preventing tile_eclipse from loading
/opt/tilera/TileraMDE-version/arch/lib/ide/jre/lib/i386/client/libjvm.so
which requires text relocation.
```

The default SELinux configuration is different for different Linux distributions. And your site administrator might have arbitrarily modified the SELinux policy to suit your site's requirements. Thus, we recommend that if you see errors or unexpected tool failures like those above, check your SELinux configuration and if necessary modify the SELinux enforcement mode and/or policy as needed.

For general information on SELinux, see the following Wikipedia article:

<http://en.wikipedia.org/wiki/Selinux>

For more information on the SELinux component of your Linux distribution, see `man selinux`.

As a general rule, it is better to add specific exceptions to SELinux for the executables and/or libraries in question, rather than reducing the overall enforcement level of SELinux. However, if you are allowed to change the SELinux enforcement level, try setting it to permissive mode temporarily, just to confirm that SELinux policy enforcement is the reason for the failure.

CHAPTER 2 DOCUMENTATION FOR THE 4.3 RELEASE

This chapter lists the documentation is available for the MDE 4.3 release:

- [Section 2.1.1 Primary MDE Software Documentation Manuals](#)
- [Section 2.1.2 TILE-Gx Processor Architecture and Hardware Manuals](#)
- [Section 2.1.3 Libraries Documentation](#)
- [Section 2.1.4 Development Tools Documentation](#)
- [Section 2.1.5 Operating System Internal Interfaces](#)
- [Section 2.1.6 Application Notes](#)

2.1 Documents Available Through the MDE Document Index

These documents are all available through the MDE Document Index, which is the `/doc/index.html` file in the MDE installation.

2.1.1 Primary MDE Software Documentation Manuals

- *Multicore Development Environment User Guide* (UG501)
- *Programming the TILE-Gx Processor* (UG505)
- *MDE mPIPE Programmer's Guide* (UG506)
- *MDE TRIO Programmer's Guide* (UG503)
- *MDE MiCA Programmer's Guide* (UG507)
- *Multicore Development Environment System Programmer's Guide* (UG509)
- *Using the Tiler IDE* (UG511)
- *Multicore Development Environment Optimization Guide* (UG515)
- *Migrating to the TILE-Gx Processor Family* (UG521)

2.1.2 TILE-Gx Processor Architecture and Hardware Manuals

- *Architecture Overview for the TILE-Gx Family of Processors* (UG420)
- *Tile Processor Architecture Overview for the TILE-Gx Series* (UG130)
- *TILE-Gx Instruction Set Architecture* (UG401)
- *Tile Processor and I/O Device Guide for the TILE-Gx Family of Processors* (UG404)
- *TILEmpower-Gx Platform User's Guide* (UG405)
- *TILExtreme-Gx Platform Start-Up and Installation Guide* (UG409)
- *TILExtreme-Gx Duo Start-Up and Installation Guide* (UG411)

- *TILEncore-Gx72 Card User's Guide* (UG414)
- *TILEncore-Gx36 Card User's Guide* (UG416)

2.1.3 Libraries Documentation

- *Application Libraries Reference Manual* (UG527)
- *Vector Library API Reference Manual* (UG217)
- "The TILE C Library"
- "The TILE C Math Library"
- "Choosing Which Libc to Use"

2.1.4 Development Tools Documentation

- TILE-Gx Processor C and C++ Compiler (`tile-gcc`, `tile-g++`)
- TILE-Gx Processor Simulator (`tile-sim`)
- Tile Processor Monitor (`tile-monitor`)
- Tile Processor Eclipse-based Integrated Development Environment (`tile-eclipse`)
- C Compiler for the TILE-Gx Classifier Processor (`tile-mpipe-cc`)
- mPIPE statistics display utility (`mpipe-stat`)
- TILE-Gx Processor Assembler (`tile-as`)
- Tile Processor Profiler (`tile-gprof`)
- TILE-Gx Processor Linker (`tile-ld`)
- TILE-Gx Processor Binary Utilities
- TILE-Gx Processor Boot Image Creator (`tile-mkboot`)
- TILE-Gx Processor Bootstream Formatter (`tile-mkrom`)
- TILE-Gx Kernel memory bandwidth profiling tool (`memprof`)
- TILE-Gx PCIe statistics display utility (`trio-stat`)
- Tile Processor HTTP-protocol document server (`tile-thd`)
- Hardwall Special File for UDN Access (`/dev/hardwall`)
- Tilera Linux low-level memory management kernel APIs: `mmap(2)`, `mbind(2)`
- Compiling for the TILE Architecture
- *Board Test Kit Guide for TILE-Gx Processors* (UG422)

2.1.5 Operating System Internal Interfaces

- *Hypervisor API Reference* (UG226)

2.1.6 Application Notes

- *Tilera Board Information Block Format* (AN040)

2.1.7 I/O Device Documentation

- *Tile Processor and I/O Device Guide for the TILE-Gx Family of Processors* (UG404)

2.2 Man Pages

2.2.1 Section 1 (General Commands) Man Pages

These man pages are available:

- opcontrol
- ophelp
- oprofile
- tile-as
- tile-c++ (brings up man page for tile-g++)
- tile-cc (brings up man page for tile-gcc)
- tile-eclipse
- tile-g++
- tile-gcc
- tile-gdb
- tile-gen-initramfs
- tile-gprof
- tile-ld
- tile-mkboot
- tile-mkrom
- tile-monitor
- tile-mpipe-cc
- tile-reboot
- tile-sim
- tile-trace-view

2.2.2 Section 2 (System Calls) Man Pages

These man pages are available

- mbind
- mmap

2.2.3 Section 4 (File Formats) Man Pages

This man page is available

- hardwall

2.2.4 Section 5 (Miscellaneous) Man Pages

This man page is available:

- pgtable

CHAPTER 3 EXAMINING THE TILERA MULTICORE DEVELOPMENT ENVIRONMENT

This chapter provides a high-level overview of the MDE:

- [Section 3.1 Overview of the MDE Components](#)
- [Section 3.2 Steps in the Development Process](#)
- [Section 3.3 Hardware and Simulator Development Platforms](#)
- [Section 3.4 Interacting with the Tile System](#)

3.1 Overview of the MDE Components

Tilera's Multicore Development Environment™ (MDE) is a set of tools that allow developers to take advantage of the parallel processing potential of the Tile Processor™ architecture.

The MDE enables developers to port large and complex sequential and parallel applications to a multicore platform.

The tools are based on industry standards, you will encounter a minimal learning curve then you will be up and running.

The primary MDE components are:

- `tile-gcc` and `tile-g++` compilers
- Standard run-time environment using full SMP Linux 3.10
- Eclipse-based Integrated Development Environment (IDE) with integrated tools for application writing, building, execution, debugging, and performance analysis
- Standard command-line tools GDB and OProfile
- Hardware and simulator development platforms for prototyping, testing, and running an application
- Tilera Multicore Components library (TMCTM), which provides many useful primitives for building parallel programs

You can perform some tasks using the IDE and others using the command-line tools, if you prefer. For example, you might prefer to do hardware profiling from the command line but view results in the IDE's **Profile** view. For information about the IDE, see [Chapter 5: Getting Started with the IDE](#).

For information about the command-line tools, see [Chapter 4: Running Programs Using the Command-Line Interface](#).

Use the `tile-monitor` tool to interact with a TILE-Gx™ processor from an x86 host. It provides a single, integrated interface for many common tasks, including creating bootroms, booting Tile devices, transferring files between the development host and the Tile environment, mounting host files into the Tile Linux filesystem, running applications, and using debugging and profiling tools. For complete details about using the `tile-monitor` tool, see Section 2.1 "What Is `tile-monitor`?" in the *Multicore Development Environment User Guide* (UG501).

The command-line interface is particularly good for:

- Batch-mode simulation and profiling
- Automated builds

3.2 Steps in the Development Process

Application source code for the Tile Processor can be written and edited using any standard text editor, or using the Eclipse-based IDE.

In general, the development process involves these steps:

- **Author** — Writing an application, or porting an existing application to run on the Tile Processor. The Tile Processor is programmed using ANSI C and C++. See *Programming the TILE-Gx Processor* (UG505) for details.
- **Build** — Compiling and building an application using the C/C++ compiler and standard Makefiles, invoked by means of the IDE or the command line.
- **Run** — Running an application in various execution environments, including software simulation and hardware development platforms. You can launch and execute applications using the IDE or the command line.
- **Debug** — Solving correctness problems in an application. The MDE provides specific support for debugging multiprocess applications on the Tile Processor architecture.
- **Evaluate/Optimize** — Measuring performance to determine where and how the application is spending time and tuning the application to improve performance. The MDE provides rich performance information and specific tools you can use to improve performance.

3.3 Hardware and Simulator Development Platforms

A Tilera application consists of one or more processes, each of which is instantiated from an executable, that run together as a unit. An application can be run on either of the following:

- Actual physical hardware (*hardware platform*)
- Simulation of the Tile Processor hardware on the host platform (*simulator platform*)

The hardware platform contains the TILE-Gx™ processor, and is one of the following:

- TILEncore-Gx9™ Intelligent Application Adapter
- TILEncore-Gx16™ Intelligent Application Adapter
- TILEncore-Gx36™ Intelligent Application Adapter
- TILEncore-Gx72™ Intelligent Application Adapter
- TILEmpower-Gx™ 1U developer platform, such as a TILEmpower-Gx36™, TILEmpower-Gx72, TILExtreme-Gx36™ or TILExtreme-Gx36™ Duo appliance
- Third-party or customer-developed platform

Running on hardware provides full-speed operation as well as source-level debugging and the gathering of essential performance analysis information.

The simulator platform is a simulation of the TILE-Gx Processor on a Linux x86 host, including the processor and caches on each tile, the iMesh™ (intelligent Mesh) interconnect, on-chip I/O devices, and memory. It provides an accurate timing model, source-level debugging, and rich performance information. You can run the simulator in cycle-accurate mode for precision, or in functional mode (`--functional`) for faster execution. You can also switch between the two modes programmatically, allowing faster execution during uninteresting code segments.

Both hardware and simulator launches boot a copy of the Tile Linux OS with its own virtual file system on the target, essentially acting as a RAM disk.

The primary difference between the two platforms is that the hardware platforms have better performance; the simulator platform has better profiling depth and detail.

3.4 Interacting with the Tile System

The easiest way to interact with a TILE-Gx processor is to use the `tile-monitor` command.

Table C shows the various `tile-monitor` command-line options for controlling interaction with the Tile system.

Table 3-1. The Effects of Various `tile-monitor` Arguments

This <code>tile-monitor</code> argument:	Allows you to:
<code>--resume</code>	Connect to the system without booting the system.
<code>--net hostname</code>	Connect to a Tile system after booting it, by doing the following: • Build a boot image (<code>tile-monitor</code> invokes <code>tile-mkboot</code>). • Copy the boot image to the Tile system over network (via <code>scp</code>). • Program it into SROM (<code>ssh</code> to run <code>sbim</code>). • Boot the image (<code>ssh</code> to execute <code>reboot</code>). • Connect to the Tile system over the network.
<code>--net hostname</code> <code>--resume</code>	Connect to the Tile system over network without booting the system.
<code>--dev USB_device_name</code>	Connect to the Tile system over USB after booting it, by doing the following: • Build a boot image (<code>tile-monitor</code> invokes <code>tile-mkboot</code>). • Push the boot image over USB. • Boot the image at <code>/dev/tileusbN/boot_device</code>. • Connect to the Tile system over USB.
<code>--dev USB_device_name</code> <code>--resume</code>	Connect to the Tile system over USB without rebooting the system.
<code>--dev PCIE_device_name</code>	Connect to the Tile system over PCIe after booting it, by doing the following: • Build a boot image (<code>tile-monitor</code> invokes <code>tile-mkboot</code>). • Push the boot image over PCIe. • Boot the image at <code>/dev/tilegxpciN/boot_device</code>. • Connect to the Tile system over PCIe.
<code>--dev PCIE_device_name</code> <code>--resume</code>	Connect to the Tile system over PCIe without rebooting the device.
<code>--dev bmc_ip_addr#N</code>	Connect to CPU# <i>N</i> on a TILExtreme-Gx system over USB?, by doing the following: • Build a boot image (<code>tile-monitor</code> invokes <code>tile-mkboot</code>). • Push the boot image to the <code>sda1</code> device of the CPU # <i>N</i> on the TILExtreme-Gx appliance over the motherboard USB hub. • Boot the image at <code>/dev/tileusbN/boot_device</code>. • Connect to the Tile system over USB.

3.4.1 Understanding Tile Boot Processing

The typical Tile device booting process involves the following activities:

1. The boot process starts with a hardware reset of the Tile Processor chip.
2. After hardware reset, each tile executes a small built-in bootstrap program (termed the *level-0 bootstrap*) which copies an initial amount of boot code from an external device to the Tile's L2 cache, and executes it.
3. The first portion of the boot image is transmitted from the device, over the iMesh™, to one specific Tile on the TILE-Gx chip that is near that I/O device. This first piece of boot code is termed the *level-1 bootstrap*.
4. The level-1 bootstrap performs basic initialization of the chip, by replicating itself from the initial boot Tile to all of the other Tiles and to fully configure the iMesh network coordinates. It also configures the memory controllers and performs some preliminary tests of the memory.
5. The level-1 bootstrap loads the second portion of the boot image (the hypervisor code) into memory, and then jumps to it.
6. The hypervisor finishes the initialization of the TILE-Gx chip. It reads the third and final part of the boot image into memory. This is the hypervisor file system, composed of the hypervisor configuration file and supervisor code.
7. The hypervisor initializes the on-chip devices, allocating dedicated tiles to manage them if needed.
8. The hypervisor loads the supervisor (the OS, usually Linux) into memory and begins executing it on one tile.
9. The supervisor initializes its own internal data structures and performs any startup operations required to configure its allocated resources.
10. The supervisor executes an initialization program or script to bring up OS services or applications.

3.4.2 Understanding Hypervisor Definition Processing

You can configure the hypervisor code using these `tile-monitor` arguments:

`--hvc hypervisor_configuration_file`

Specifies a custom hypervisor configuration file to use in this invocation of the `tile-monitor` command. You may supply multiple configuration files by using multiple instances of the `--hvc` switch.

`--hvd hypervisor_definition`

Specifies a hypervisor per-processor definition, which lets you override a setting in a hypervisor configuration file.

`--hvx supervisor_definition`

Lets you concatenate a specification onto a special hypervisor definition. Tilera uses this to specify supervisor (ie. Linux) boot arguments.

The hypervisor has a pre-processor function which evaluates `--hvd` definitions to supply missing values, override values in the configuration file or to evaluate conditional statements in the hypervisor configuration file.

Note: From Linux, you can view the post processed hypervisor configuration information by examining the file `/sys/hypervisor/hvconfig`.

Figure 3-1 illustrates how arguments to the `tile-monitor` command affect and control Linux hypervisor and supervisor customization.

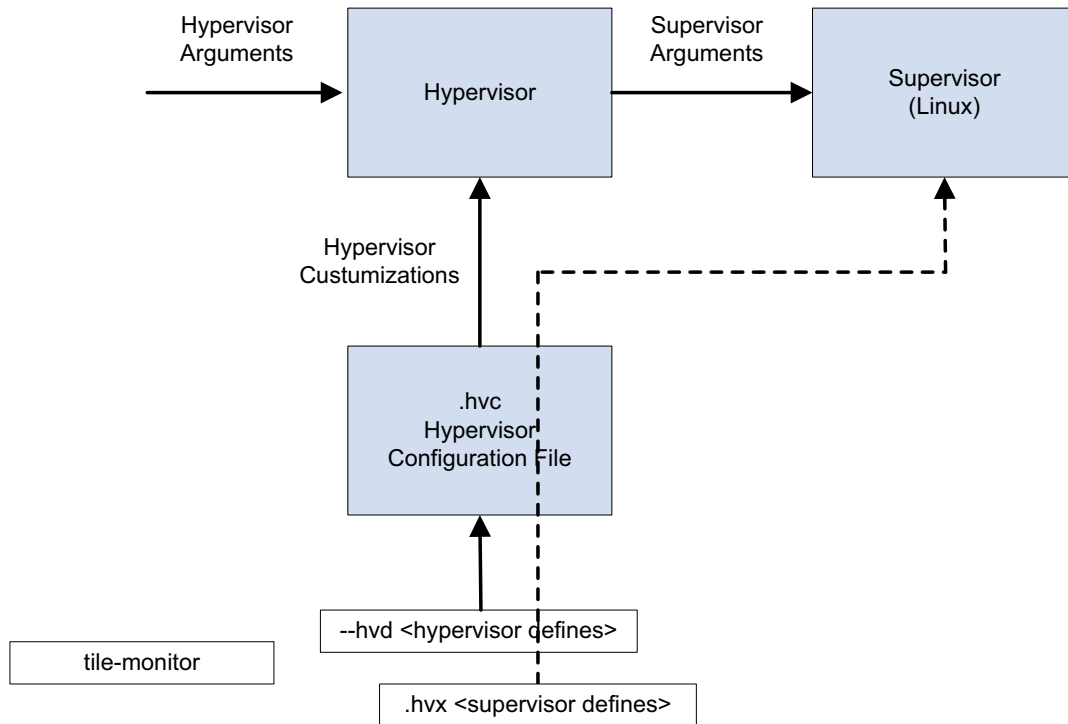


Figure 3-1: Linux Hypervisor and Supervisor Customizations

CHAPTER 4 RUNNING PROGRAMS USING THE COMMAND-LINE INTERFACE

This chapter describes:

- [Section 4.1 Overview](#)
- [Section 4.2 Getting Help on the MDE Commands](#)
- [Section 4.3 Before You Use the Examples](#)
- [Section 4.4 Running the Example Programs on the Hardware Platform](#)
- [Section 4.5 Running the Example Programs on the Simulator](#)
- [Section 4.6 Explanation of Output](#)
- [Section 4.7 Where to Go from Here](#)

4.1 Overview

The installation includes single-tile and multi-tile examples for both the hardware/network-attached systems and simulator platforms. This chapter describes how to use the supplied makefiles, compiler, and `tile-monitor` tool to run the examples.

These examples assume that you have set up your environment as described in [Section 1.4 “Installing the Full MDE Software on a Linux x86 Host”](#) on page 3.

You can use makefiles to drive development as you would normally do; no special environment configuration is needed, aside from including the MDE `bin` directory in your `PATH`.

You can use traditional command-line development and debugging techniques, and the MDE includes familiar tools to support this (compilers, debuggers, and so forth).

4.2 Getting Help on the MDE Commands

The commands in the MDE command-line interface generally follow the Linux convention of providing a `--help` option that summarizes the command’s usage. For example, to see the help for the `tile-gcc` command:

```
$ tile-gcc --help
```

The `tile-monitor` tool has several kinds of help:

```
$ tile-monitor --help-examples
$ tile-monitor --help-commands
$ tile-monitor --help-options
$ tile-monitor --help-all
$ tile-monitor --help-stdin
```

The commands also have associated man pages that describe how to use them and what options are supported. You can configure your standard `man` command to allow searching the MDE manual pages. Tiler’s manual pages are found under `$TILER_ROOT/man`. To configure the `man` command to display our manual pages, you need to do either of the following:

- Edit the `/etc/man.config` file for your site to include our manpage directory.
- Alternatively, set or edit the `MANPATH` environment variable, either in a global or a user-specific shell init script.

Refer to the manpage on `man` itself, which provides more details on both methods.

Two manpages you might want to read are:

```
$ tile-man tile-gcc
$ tile-man tile-monitor
```

Note: Most of the commands also have HTML files in the Tiler MDE Document Index
`$TILER_ROOT/doc/index.html`.

4.3 Before You Use the Examples

The examples described here are provided in these directories:

```
$TILER_ROOT/examples/getting_started/hello_world
$TILER_ROOT/examples/getting_started/hello_world_pthread
```

Copy the directories to your own working directories and use those instead. For example:

```
$ mkdir ~/examples
$ cp -r $TILER_ROOT/examples/getting_started/hello_world ~/examples/
$ cp -r $TILER_ROOT/examples/getting_started/hello_world_pthread ~/examples/
```

To run the examples that follow, go to your `hello_world` directory:

```
$ cd ~/examples/hello_world
```

and your `hello_world_pthread` directory:

```
$ cd ~/examples/hello_world_pthread
```

4.4 Running the Example Programs on the Hardware Platform

In the examples below, for the hardware platform we use the `run_dev` target defined in the example makefiles.

4.4.1 Running the Single-Tile Example Program

To run the single-tile example program on the TILE-Gx device:

1. `cd` to the `hello_world` directory.

2. Use the `run_dev` target of the provided makefile to build the example and run it on the hardware:

```
$ make run_dev
```

Output similar to the following is displayed:

```
$TILERA_ROOT/bin/tile-monitor \
  --functional \
  --image 2x2 \
  --mkdir /opt/test --cd /opt/test \
  --upload hello_world hello_world --tile 2x2 -- hello_world
Hello World!
```

4.4.2 Running the Multi-Tile Example Program

To run the multi-tile example program on the TILE-Gx device:

1. `cd` to the `hello_world_pthread` directory.
2. Use the `run_dev` target of the provided makefile to build the example and run it on the hardware:

```
$ make run_dev
```

Output similar to the following is displayed:

```
$TILERA_ROOT/bin/tile-monitor \
  --image 4x4 \
  --mkdir /opt/test --cd /opt/test \
  --upload hello_world hello_world --tile 4x2 -- hello_world
Main process: Hello world!
Thread on tile 1: Hello, Brave New World!
Thread on tile 0: Hello, Brave New World!
Thread on tile 2: Hello, Brave New World!
Thread on tile 3: Hello, Brave New World!
Started 4 pthreads
```

Because `stdout` captures the output independently for each process, it might not be displayed in the order shown. You should, however, see one `Hello..` and one `Hello Again...` message for each process.

See [Section 4.6 “Explanation of Output” on page 30](#) for a discussion of what the output means.

4.5 Running the Example Programs on the Simulator

The examples below use the `run_sim` target, which runs the example using the Tileria® simulator. These examples can be run on any host where the MDE is accessible, and do not require a TILE-Gx device.

4.5.1 Running the Single-Tile Simulator Example Program

To run the single-tile example program on the simulator platform:

1. `cd` to the `hello_world` directory.

2. Use the `run_sim` target to build the example and run it on the simulator:

```
$ make run_sim
```

Output similar to the following is displayed:

```
$TILERA_ROOT/bin/tile-monitor \
  --functional \
  --image 2x2 \
  --mkdir /opt/test --cd /opt/test \
  --upload hello_world hello_world --tile 2x2 -- hello_world
Hello World!
```

4.5.2 Running the Multi-Tile Simulator Example Program

To run the multi-tile example program on the simulator platform:

1. `cd` to the `hello_world_pthread` directory.
2. Use the `run_sim` target to build the example and run it on the simulator:

```
$ make run_sim
```

Output similar to the following is displayed:

```
$TILERA_ROOT/bin/tile-monitor \
  --image 4x2 \
  --mkdir /opt/test --cd /opt/test \
  --upload hello_world hello_world \
  --tile 4x2 -- hello_world
Main process: Hello world!
Thread on tile 1: Hello, Brave New World!
Thread on tile 0: Hello, Brave New World!
Thread on tile 2: Hello, Brave New World!
Thread on tile 3: Hello, Brave New World!
Started 4 pthreads
```

Because `stdout` captures the output independently for each process, it might not be displayed in the order shown. You should, however, see one `Hello..` and one `Hello Again...` message for each process.

See [Section 4.6 “Explanation of Output” on page 30](#) for a discussion of what the output means.

4.6 Explanation of Output

The list below shows what `tile-monitor` does in the examples.

Note that some of these steps are specific to the TILE-Gx device, and some to the simulator; the overall sequence is the same for both platforms, however.

1. Connects to the target:
 - Connects to the TILE-Gx device and boots it (`--dev` option; hardware platform only)
 or:
 - Requests a 2x2 or 4x2 simulated chip (`--image` option; simulator platform only)
2. Enables functional mode, which is faster than the default timing-accurate mode (`--functional` option; simulator platform only).

3. Creates a directory called `/opt/test` in the Linux virtual file system on the hardware or simulator target (`--mkdir` command).
4. Makes the `/opt/test` directory the current working directory (`--cd` command).
5. Uploads `hello_world` from the source to the target (`--upload` command).
6. Specifies that the application should run on a limited set of tiles (`--tile` command).
7. Runs the `hello_world` program. The space after the two dashes; the “`--`” argument indicates that `tile-monitor` should run the given program and then exit.

4.7 Where to Go from Here

To learn more about using the MDE:

- See the *Multicore Development Environment User Guide* (UG501).
- Look at the man pages and HTML documentation for important MDE command-line tools such as the:
 - `tile-gcc` and `tile-g++` compilers
 - Tiler simulator (`tile-sim`)
 - OProfile profiling support

CHAPTER 5 GETTING STARTED WITH THE IDE

This chapter describes the IDE:

- [Section 5.1 Overview](#)
- [Section 5.2 Bringing Up the Workbench for the First Time](#)
- [Section 5.3 Getting Help on the Tiler IDE](#)
- [Section 5.4 About the Eclipse UI and Its Terminology](#)
- [Section 5.5 Tiler Extensions to Eclipse](#)
- [Section 5.6 Using the Tutorial to Become Familiar with the Graphical User Interface](#)
- [Section 5.7 Creating a Project](#)
- [Section 5.8 Building the Project](#)
- [Section 5.9 MDE Execution/Monitoring Framework](#)
- [Section 5.10 Running an Application](#)
- [Section 5.11 Running an Application Using a Launch Configuration](#)
- [Section 5.12 Continuing the Tutorial](#)

5.1 Overview

The Tiler® Integrated Development Environment (IDE) provides an integrated environment for developing, debugging, and profiling applications on Tiler’s hardware and simulator platforms.

The IDE is based on the Eclipse development environment. Tiler’s IDE extends Eclipse by adding the CDT (C Development Toolkit) plug-in, plus a number of Tiler proprietary extensions to both Eclipse and CDT.

Note: This chapter provides an introduction to the IDE. For more information, see the [Tiler IDE Online Documentation](#).

5.2 Bringing Up the Workbench for the First Time

To start the IDE, enter this command at the operating system prompt:

```
$ tile-eclipse
```

Note: Always invoke `tile-eclipse` from your current MDE release directory. Invoking `tile-eclipse` from an earlier or later MDE release is not supported.

The first time you run `tile-eclipse`, the software detects that you are running the IDE for the first time (see [Figure 5-1](#)) and automatically loads a number of perspectives for you to use in developing applications.

After you do this, `tile-eclipse` offers to display the online help browser for you. It is recommended that new users at least read through the “Getting Started with the IDE” section of the online help, because this provides a walk-through of Tilera-specific features of the IDE.

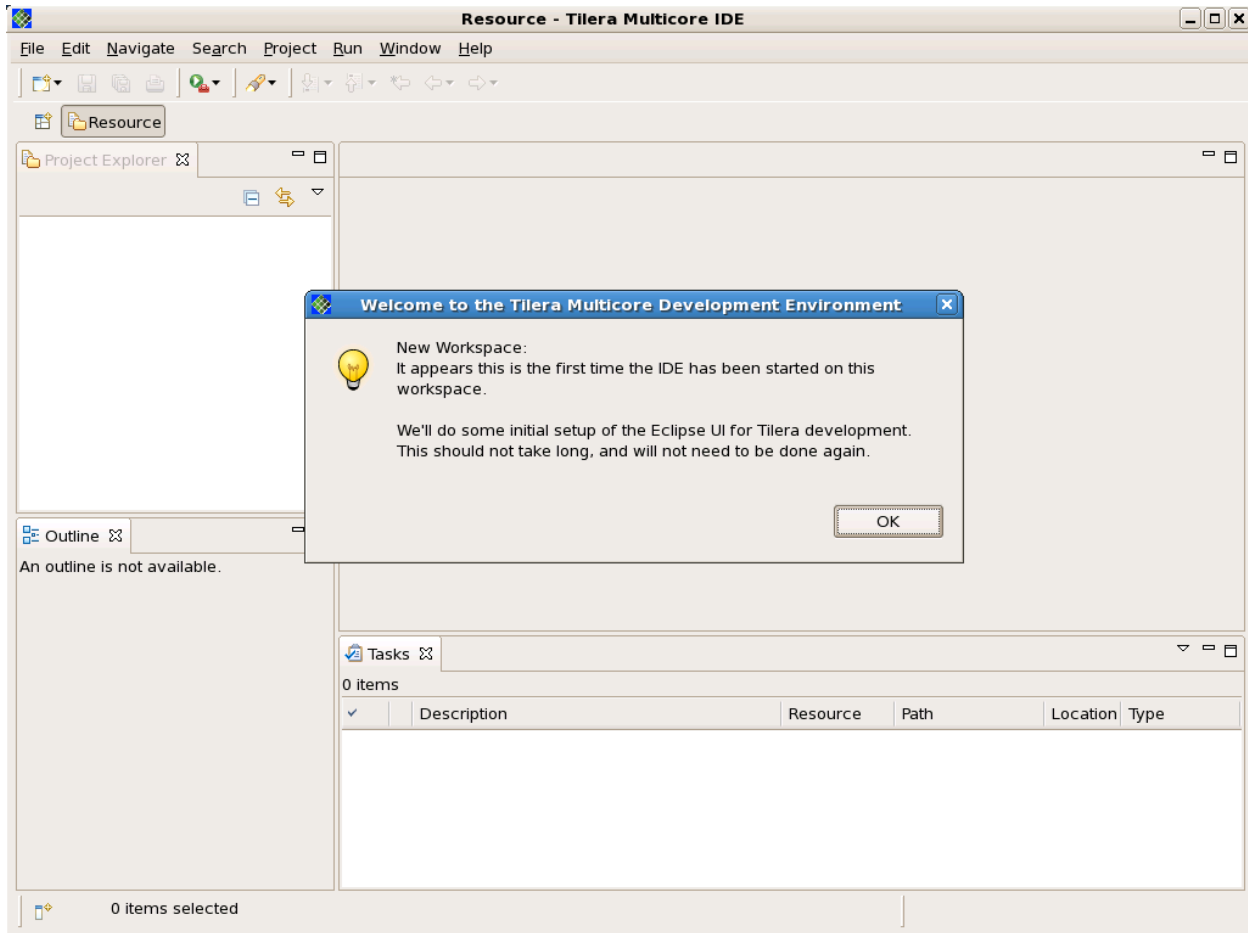


Figure 5-1: Tlera IDE When It Is Run the First Time

Once this initial setup is completed, the IDE is ready for you to use in developing, running, and profiling applications.

5.3 Getting Help on the Tilera IDE

You can access help for the Tilera IDE in two ways:

- By using the online help browser (see [Section 5.3.1 “Using the Online Help Browser”](#) on page 35)
- By using context-sensitive help (see [Section 5.3.2 “Context-Sensitive Help”](#) on page 36)

Note: There is some additional information about the IDE command-line options in the `tile-eclipse.html` page in the MDE Document Index. The page is called “Tile Processor Eclipse-based Integrated Development Environment (`tile-eclipse`)” in the “Development Tools” section.

5.3.1 Using the Online Help Browser

To access the online help browser from within the IDE, select **Help → Help Contents** to display the **Help Window**. In the **Contents** pane, the following headings are presented:

- **Workbench User Guide**
- **C/C++ Development User Guide**
- **Tilera IDE Online Documentation**

Double-click on **Tilera IDE Online Documentation** in the **Contents** pane to bring up the IDE online documentation as shown in [Figure 5-2](#).

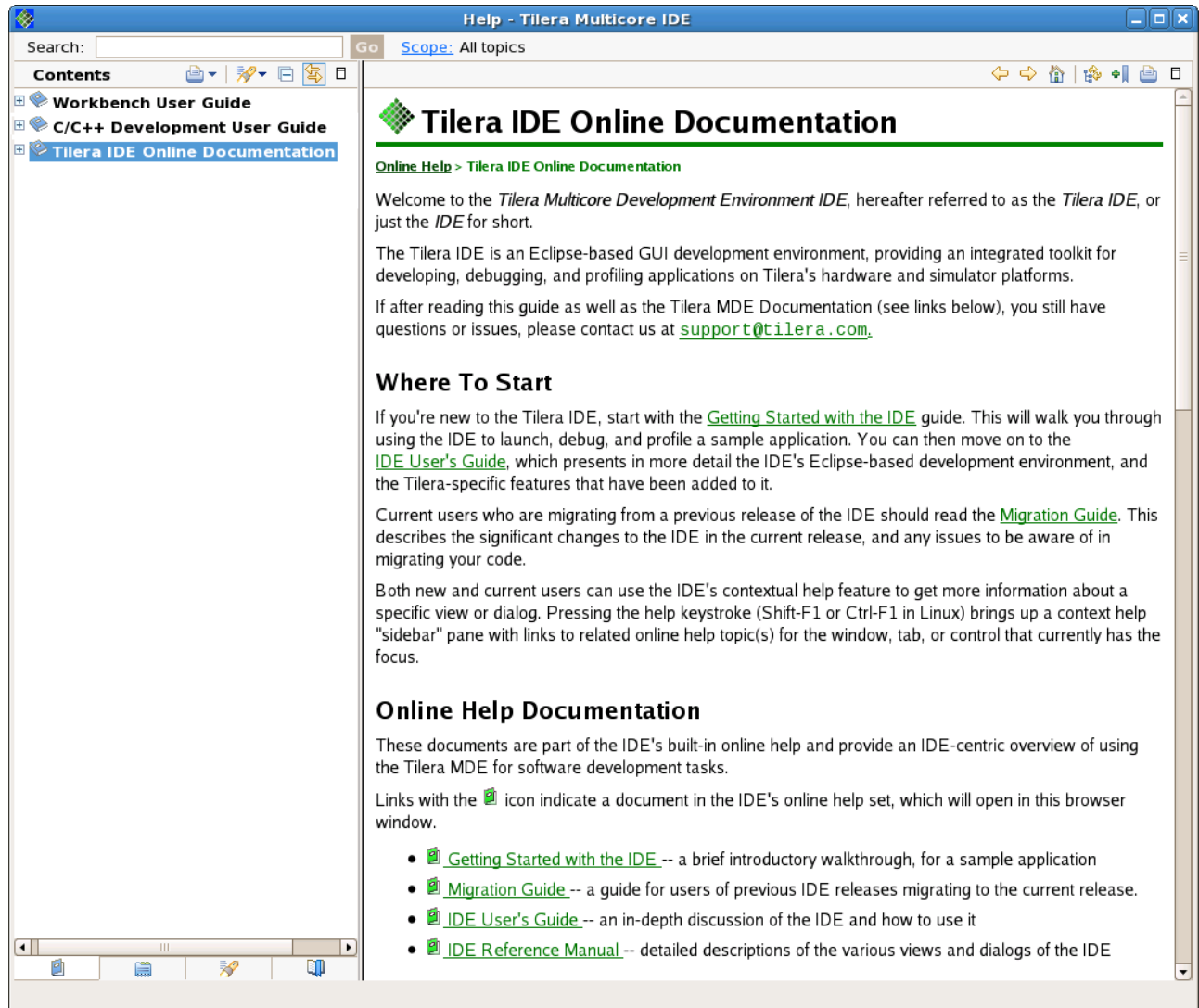


Figure 5-2: Online Help Browser Window

The **Tilera IDE Online Documentation** provides detailed documentation about the Tilera-specific features of the IDE. The online help is divided into three sections:

- “Getting Started with the IDE”: a brief introductory walkthrough, using an example application

- “IDE User’s Guide”: an in-depth discussion of the IDE and how to use it
- “IDE Reference Manual”: detailed descriptions of the views and dialogs of the IDE

The following non-Tilera help documents are also available:

- **Workbench User Guide**: Explains the workbench, perspectives, editors, views, external tools, team programming, and accessibility features.
- **C/C++ Development User Guide**: Documents the C/C++ Development Toolkit (CDT), which is a collection of Eclipse-based features that provides the capability to create, edit, navigate, build, and debug projects that use C or C++ as a programming language.

5.3.2 Context-Sensitive Help

You can get assistance with the view, tab, or control that currently has the focus by typing the Help keystroke. In Linux, this is **Shift-F1** or **Control-F1**.

Invoking context-sensitive help brings up a sidebar **Help** view at the right of the current window. For example, if you click anywhere in the **Monitor View** to highlight it, and then type **Control-F1**, the help view appears. See [Figure 5-3](#).

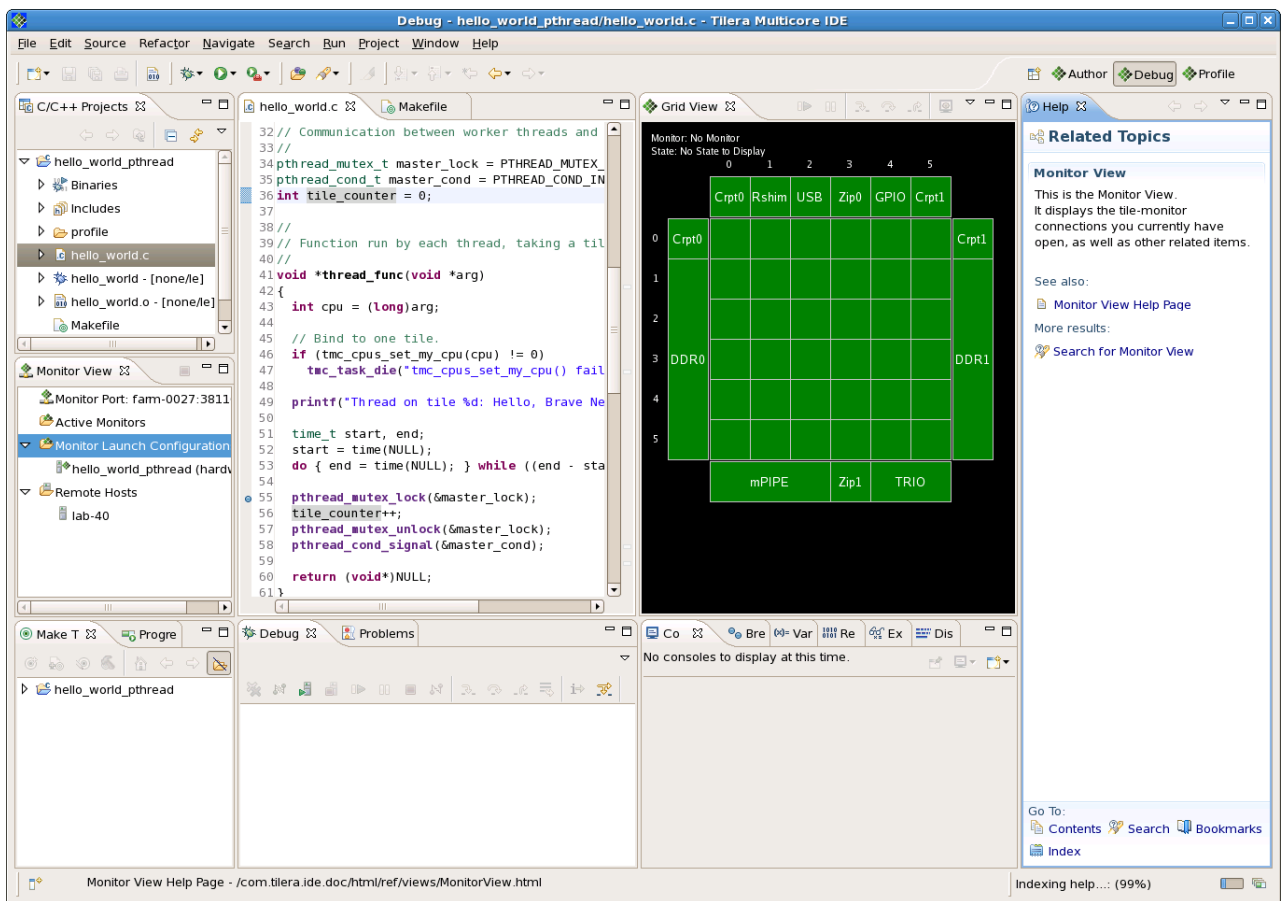


Figure 5-3: Help View Shown at Right of Window

In this case, the **Help** view displays the selected view (the **Monitor View**) and a brief summary of its purpose. It also displays a list of related online help topics that you can click on for more detail. See [Figure 5-4](#).

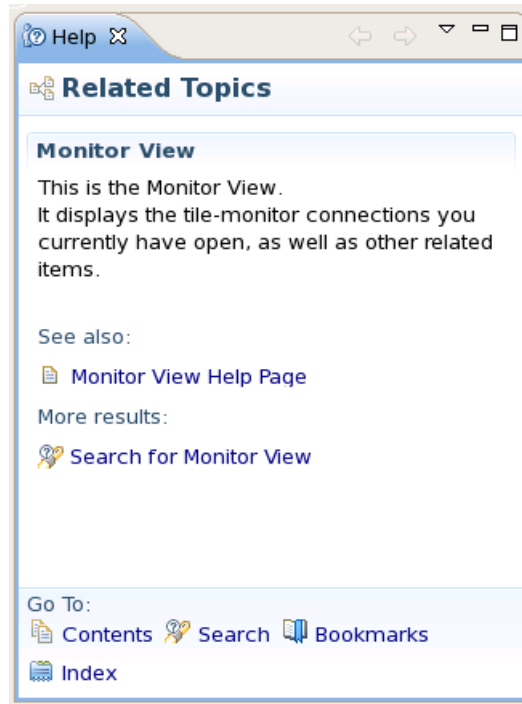


Figure 5-4: Help View

The underlying Eclipse IDE provides contextual help for its own views and controls. Thus, depending on where the focus is, the topics displayed might not be Tileria-specific. If you do not see the help you need, try selecting a more specific tab or control, then try invoking contextual help again.

Also, be aware that you might have the focus set too narrow: If you want help for a tab, rather than a control on that tab, make sure the focus is not within a control on the tab, like a textbox. If all else fails, you can browse to the online help for the control, then go up one level to see the help for the tab.

5.4 About the Eclipse UI and Its Terminology

The Tileria IDE is based on Eclipse, and makes use of a number of Eclipse concepts, such as views, perspectives, and so on. The basic terminology that you need to know in order to use the Tileria IDE is described in the sections that follow.

If you have not used an IDE before, or having not used the Tileria IDE before, you might want to browse through an introductory text on Eclipse, such as *Eclipse Distilled* by David Carlson.

5.4.1 Workspace Directory

A *workspace directory* is simply a “default” root directory that Eclipse uses as a location for projects and the preferences, settings, and project-related metadata. When you start up the Tileria IDE, a workspace directory is created for you in the following location:

```
${HOME}/.tilera/workspace
```

If you want to use a different location as the workspace directory, you can set the environment variable `TILERA_WORKSPACE` before running `tile-eclipse`. In general, this is not necessary. But you might want to do this in order to create distinct workspace directories with different releases of a group of related projects.

The “Default” MDE installation means the installation from which `tile-eclipse` was invoked.

5.4.2 Project Directory

A *project directory* is a container for the sources, makefiles, and so forth of a single application. It is also used by Eclipse to store some IDE metadata, such as the project description and any project-specific settings.

When you create a new project, the **New Project** dialog allows you to choose the location for the project. The default location is a subdirectory under the workspace directory. For example, if the project is named “my_project”, the default location would be:

```
${HOME}/.tilera/workspace/my_project
```

A project whose source directory is a subdirectory of the workspace directory is an *internal project*.

You can define the project’s location to be some other external directory, for example, if you have an existing directory with source files, makefile, and so on. When the source directory for a project is located outside the workspace, the project is called an *external project*.

Restrictions

Eclipse uses the workspace directory to store metadata about your preferences, the projects you have created, and so forth. Thus, it treats the workspace directory itself as a special case: you can create an internal project as a subdirectory of the workspace, or you can create an external project in an external directory outside the workspace. You cannot, however, have a project whose top-level directory is the same as the workspace directory itself (for either internal or external projects). Also, you cannot create an “external” project in a subdirectory of the workspace.

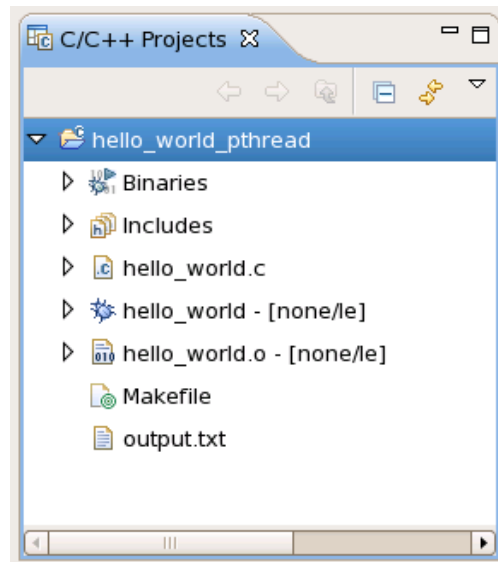
This means, for example, that if you want to use the `--here` command-line argument for `tile-eclipse` to ask `tile-eclipse` to use the current directory as the workspace, you should not do so in the root directory of your project source tree. Instead, you should move up one directory level, effectively making the parent directory of your source tree the “workspace” for your project.

5.4.3 Views

A *view* is a specific window in the IDE, such as the **Debug** view or the **Tilera Grid** view. A view typically has a tab, and the name of the view appears on the tab.

Eclipse has a large number of views for editing, debugging, and for displaying information. In general, a view is a sub-window of the Eclipse workbench with its name displayed in a tab. These include text editors, project browsers, and debugging information panes.

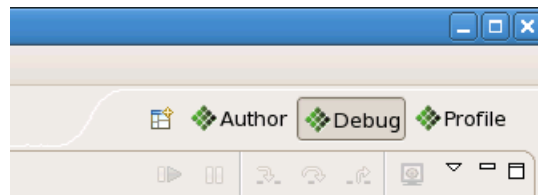
For example, here is the **C/C++ Projects** view, which displays the projects you have defined.



5.4.4 Perspectives

Views are organized into selectable groups called *perspectives*. A *perspective* is a named set of views and editors that serves a common purpose, such as the **Tilera Debug perspective** or the **Tilera Profile perspective**.

The Tilera IDE includes a couple of perspectives that select groups of Tilera-specific views: the **Author**, **Debug**, and **Profile** perspectives. You can select these by clicking the perspective's name on the perspective bar, located in the upper right corner of the IDE workbench.



5.4.5 Projects

A *project* in Eclipse is much like projects in other IDE environments: it is a folder that contains a group of files related to the construction of an application. A project is a description in the IDE of the location of the application, how to build it, and any preferences you have specified. In the **C/C++ Projects** view shown above, you can see a `hello_world` project that includes a number of files, such as a source file, a binary executable, and a makefile.

5.5 Tilera Extensions to Eclipse

The Tilera MDE adds several user-interface elements to the Eclipse IDE:

- Three Tilera-specific perspectives:
 - *Author perspective*: for authoring Tilera applications.
 - *Debug perspective*: for running and debugging code.
 - *Profile perspective*: for profiling code and examining the results of profile runs.

- There is a single Tile Monitor Launch configuration, which combines the options for target and application.
- Three Tilera-specific views:
 - *Tilera Grid view*: displays the state of the Tile Processor™ for the currently selected launch, if any.
 - *Tilera Profile view*: displays and explores the content of profile XML data files generated by applications running on the Tilera hardware or simulator.
 - *Tilera Monitor view*: displays the set of `tile-monitor` instances that have been invoked from or are being managed by the IDE, as well as any Tile Monitor Launch configurations you have defined.

5.6 Using the Tutorial to Become Familiar with the Graphical User Interface

We recommend that new users begin with the “Getting Started with the IDE” section in the [Tilera IDE Online Documentation](#). If you prefer to read the information in this guide, the rest of this chapter provides introductory information in the tutorial.

If you have not already done so, read [Section 5.4 “About the Eclipse UI and Its Terminology” on page 37](#) to familiarize yourself with concepts regarding workspace directories, project directories, views, and so forth.

Note: This tutorial assumes that you have access to a networked device that contains a Tilera chip.

5.6.1 Applications and Programs

A *program* is a binary executable that serves some purpose in a multiprocess application, such as being a producer or consumer of data, or one of a collection of workers that handle tasks dispatched by a “master” task-balancing process. The word *program* also refers to the source file(s) that are used build such an executable.

An *application* is a collection of such programs that serves some overall, self-contained goal, like processing of video data streams, inspection of network packets, and so forth.

Applications can have a variety of structures, based on how their constituent program(s) are arranged and interact. Examples include the producer-consumer and master-worker applications mentioned above.

The Tilera IDE supports applications with single or multiple programs. In the example in the tutorial, for illustration purposes we show a simple application consisting of a single program.

5.6.2 “Hello World” Application

“The Sample Application” topic in the [Tilera IDE Online Documentation](#) provides a tutorial showing how to launch, debug, and profile a small example application. It uses the “Hello World” application (`hello_world_pthread`) that is available in the software package at this location:

```
$TILERA_ROOT/examples/getting_started/hello_world_pthread
```

This program demonstrates how to write a multithreaded application that runs on multiple tiles on the Tilera hardware. You are not required to use threads in your application. This example application is used because it demonstrates how to run a program on multiple tiles. It also shows some features of the IDE that support debugging of applications that happen to use threads.

5.6.3 Copying the Examples Directory

Because the Tiler MDE installation directory might be installed read-only, we suggest creating a private copy of the example program. For example:

```
$ cd /home/scratch/ide
$ mkdir projects
$ cd projects
$ cp -r $TILER_ROOT/examples/getting_started/hello_world_pthread .
$ ls hello_world_pthread
```

```
hello_world.c Makefile output.txt
```

In the above, we assume the environment variable `TILER_ROOT` is set to the root directory of your Tiler MDE installation. See [Section 1.4 “Installing the Full MDE Software on a Linux x86 Host” on page 3](#) for an easy way to set this environment variable.

5.7 Creating a Project

If you want to use the IDE to build and manage your source directory, you need to create a project.

You are not required to use the IDE to build your application: you can also build the application outside the IDE and use the IDE only for debugging and/or profiling. However, creating projects gives the IDE more information about the location of source files for your programs, and allows it to automate some tasks, like automatically rebuilding your application when a file has been edited.

You also have a choice about where you keep the sources for your application. For simple applications, you might choose to create an internal project, with its directory located under the Eclipse workspace directory, and populate the project with source files.

Or you can create an external project that references an existing directory in some other location, which might already contain source files.

Both types of projects are displayed in the IDE’s **C/C++ Projects** view. And both types of projects are managed in the same way by the IDE.

It is common for IDE users to want to apply the IDE to development and/or debugging of pre-existing code, so we walk you through creating an external project for our example application.

5.7.1 Using the Example Application to Create a Project

If you have set up the example application described in [Section 5.6.3 “Copying the Examples Directory”](#), you can create a project in the IDE as follows (see [Figure 5-5](#)):

- Right-click the **C/C++ Projects** view
- Select **New → C Project**

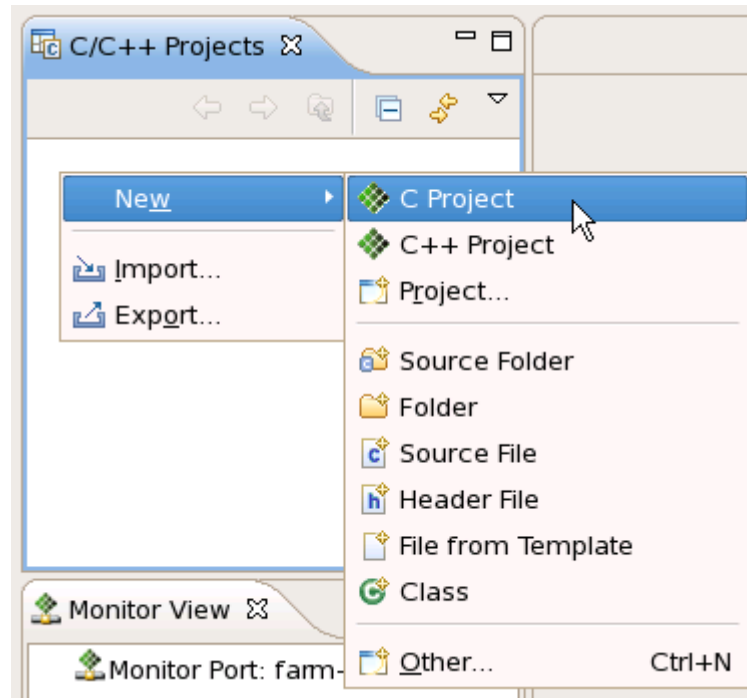


Figure 5-5: Creating a C Project

This displays the **New C/C++ Project Wizard**, shown in [Figure 5-6](#).

C Project

Please select a location and build method for this project.

Where would you like to create this project?

☐ In a subdirectory of the IDE workspace named:

hello_world_pthread

☒ In a new/existing directory outside the workspace:

Please select a directory for this project.

/home/scratch/ide/projects/hello_world_pthread Browse...

How do you want to build the project?

☒ Makefile: Your own makefile(s), or other custom build method

☐ Managed: IDE generates makefile(s) for project sources

What is the output of this managed project?

☒ An executable

☐ A shared library (.so)

☐ A static library (.a)

Which platform is this project intended for?

☒ Tilera hardware/simulator (cross-compiled with Tilera C/C++ compiler)

☐ Linux/x86 platform (compiled with standard Linux gcc)

Which Tilera MDE installation should be used?

Tilera MDE Installation Default ▼ Edit...

Click Finish to create this project with the default settings,
or click Next to edit the project's name, type, and other settings.

? < Back Next > Cancel Finish

Figure 5-6: New C/C++ Project Wizard Settings for the Example Application

The first page of the wizard contains the most commonly used settings for most applications. For most projects, you do not need to use any of the other pages.

To configure the project for the example application, all you need to do is:

1. Click the button for a new or existing directory outside the workspace
2. Enter the pathname of the example application directory

As the example application includes its own pre-existing makefile, use the default **Makefile** option for building the project. We run this application on Tiler hardware, so use the default **Tiler hardware/simulator** platform option.

The **Default** option for the **Tiler MDE Installation** is fine for this project. (This is simply the MDE installation that `tile-eclipse` itself was invoked from.)

The New Project Wizard is described in detail in the “New C/C++ Project Wizard” topic in the “IDE Reference Manual” in the **Tiler IDE Online Documentation**.

5.7.2 Closing the New Project Wizard

After your project settings are configured as described above, click **Finish** to close the wizard and create the project.

This adds a project named `hello_world_pthread` to the **C/C++ Projects** view. Click the small triangle to the left of the project name to expand it. The view should then look like this (see [Figure 5-7](#)):

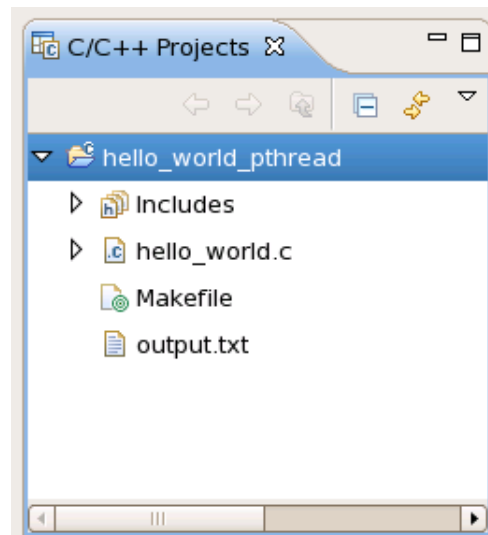


Figure 5-7: Hello World Project in Projects View

5.8 Building the Project

Because this is a Makefile project, which includes its own makefile, you can invoke the appropriate build rule from the **Make Targets** view.

The **Make Targets** view lists all of the Makefile projects you have defined, and under each project name lists the make targets that the IDE can locate.

When we created the project for the `hello_world_pthread` application, the IDE automatically examined the makefile in the application’s top-level directory and created targets for likely-looking rule names (like `all`, `build`, `run`, `debug`, and so forth).

Click the small triangle to the left of the project name in the **Make Targets** view to expand it. You should then see a list of targets something like [Figure 5-8](#):

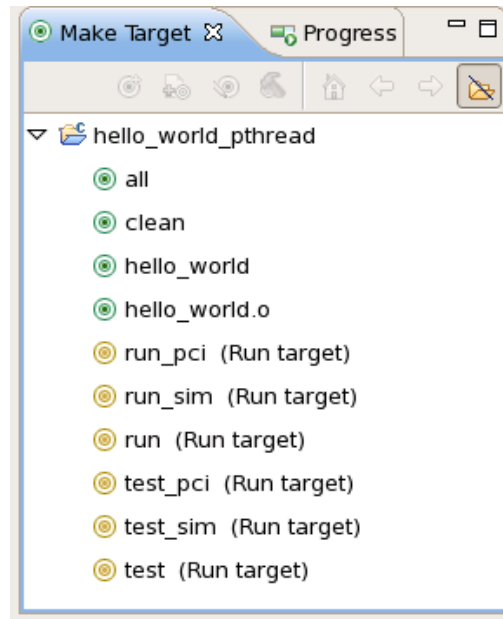


Figure 5-8: Hello World Project in Make Targets View

To invoke a make target, you can double-click its name. For example, double-click the **clean** target to ensure that we are doing a build from scratch. Then double-click the **all** target to build the application.

The output of the build process appears in the IDE's **Console** view shown in Figure 5-9.

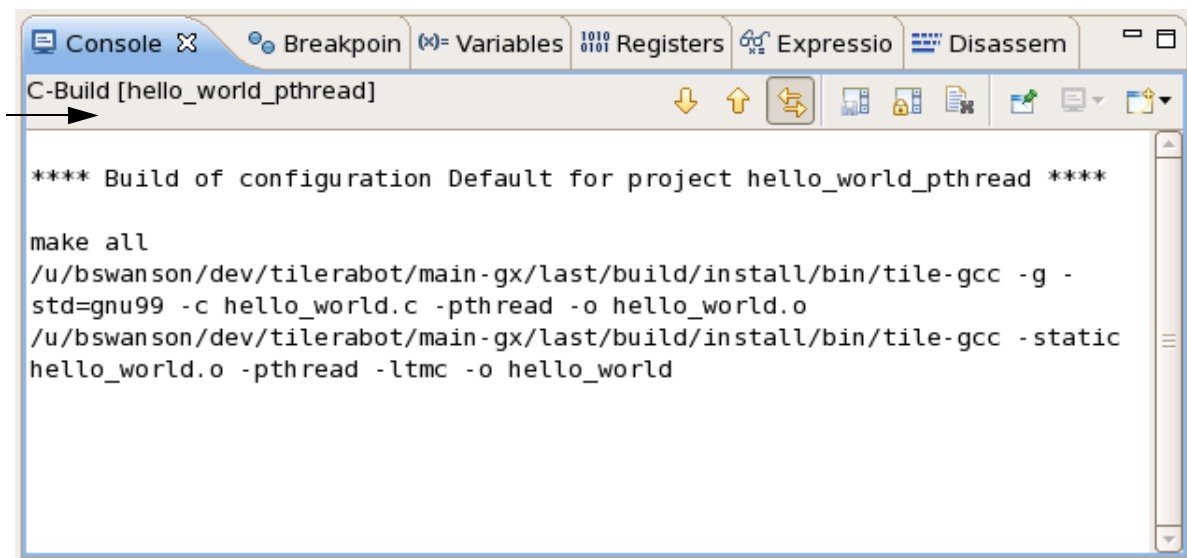


Figure 5-9: Build Output in the Console View

The **Console** view can display multiple pages of console output. The text label in the upper left corner of the view (C-Build [hello_world_pthread]) indicates which page is currently visible.

If you look at the **C/C++ Projects** view again, it shows the generated `.o` and binary executable files, as shown in [Figure 5-10](#):

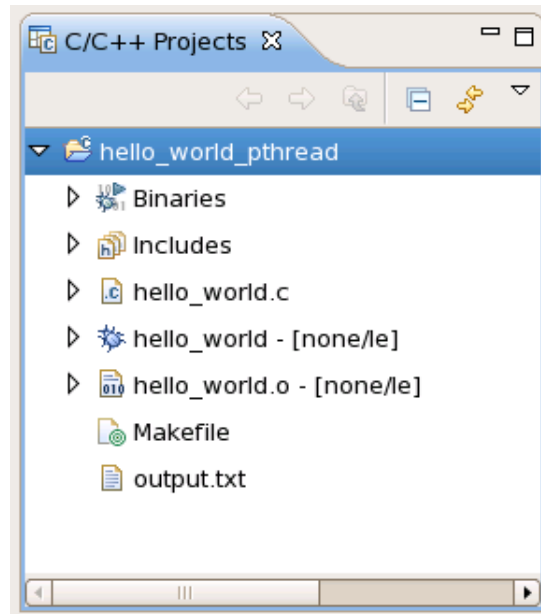


Figure 5-10: Hello World Project, After Build

In general, the **C/C++ Projects** view automatically updates after a build. If you add or remove files from the project directory from the command line, however, the view does not automatically update. To force an update, select the project name in the **C/C++ Projects** view and then do either of the following:

- Right-click and select **Refresh** from the context menu, or
- Type **F5**, the equivalent keyboard shortcut

5.9 MDE Execution/Monitoring Framework

The Tilera IDE interacts with a command-line tool, `tile-monitor`, to launch, monitor, and debug applications. The `tile-monitor` utility communicates with a `shepherd` process running on the Tile Processor. The `shepherd` manages processes/threads created by your application and reports their status to `tile-monitor`, which in turn relays it to the IDE. See [Figure 5-11](#).

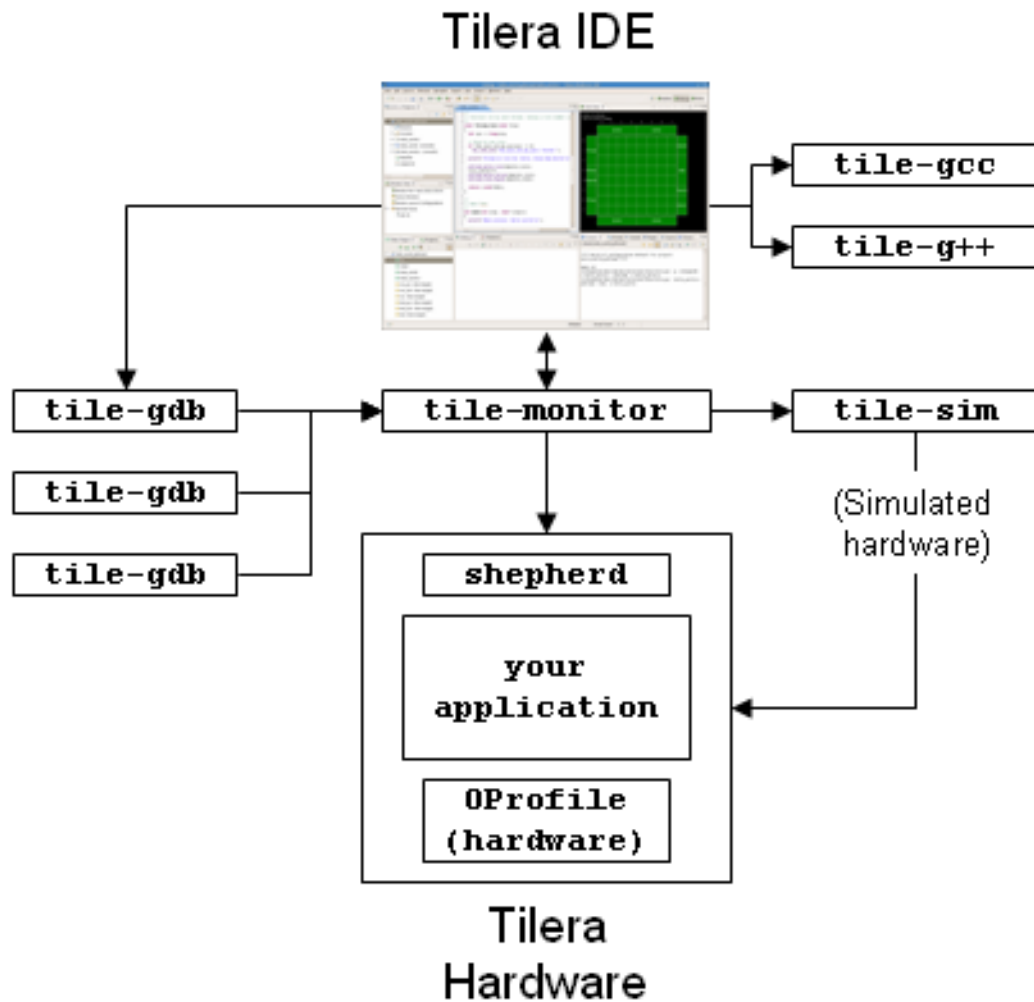


Figure 5-11: MDE Execution/Monitoring/Debugging Framework

You can also instruct `tile-monitor` to launch your application on simulated hardware. In this case, `tile-monitor` invokes the Tiler simulator, `tile-sim`, to simulate the hardware. In most respects, running on the simulator is like running on the hardware, except for execution speed. The `tile-monitor` communicates with a `shepherd` process on the simulated hardware to manage and debug applications.

You can use `tile-monitor` to launch a program from the command line without the IDE. In this case, `tile-monitor` assumes that all interaction takes place at the command line prompt. When a process needs to be debugged, `tile-monitor` displays a set of instructions for manually launching, connecting, and configuring an instance of the debugger, `tile-gdb`, to debug the program.

You can also instruct `tile-monitor` to connect to a running instance of the IDE. In this case, `tile-monitor` routes output to the IDE for display in the **Console** view. When a process needs to be debugged, `tile-monitor` communicates this to the IDE, which automatically launches `tile-gdb` instances as needed to debug the program, and manages the display of their status and output in its various debugging views.

The `tile-monitor` process connects to the IDE by means of the IDE's *monitor port*, which is basically a socket port that the IDE opens in order to listen for connections from `tile-monitor` instances. The IDE displays the Monitor Port "hostname:port" information on the first line of the **Monitor View**, as shown in [Figure 5-12](#):

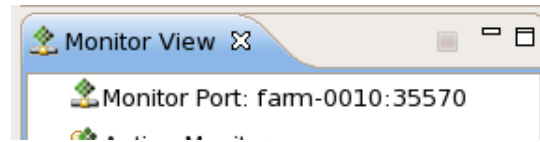


Figure 5-12: IDE Monitor Port Information in Monitor View

5.10 Running an Application

To run an application using the IDE, you have two options:

- You can invoke the application within the IDE by using a launch configuration to specify the application to run, where to run it, and any needed arguments/options.

In this case, the IDE invokes `tile-monitor` on your behalf, receives a connection from it, and then displays/debugs the application.

- You can invoke the application outside the IDE, for example by invoking a makefile rule at the command line, and use the IDE to monitor or debug the running application. You can also use the **Make Targets** view in the IDE to invoke the makefile rule from within the IDE.

Here the makefile target invokes `tile-monitor` with appropriate environment settings to tell it to connect to a running instance of the IDE. Once `tile-monitor` is connected to the IDE, display and debugging of the application is the same as when using a launch configuration above.

5.11 Running an Application Using a Launch Configuration

To run your application from within the IDE, you need to create a launch configuration. A launch configuration is a description of what you want to run and how to run it. From this, the IDE can generate a `tile-monitor` command line to launch the application.

You only need to provide the path names and options for your application. The IDE takes care of adding the necessary settings needed to instruct `tile-monitor` to connect back to the IDE.

This method can be the simplest for new users, because it automates much of the process of configuring `tile-monitor` appropriately for your application. It can also be the easiest way to get a simple test application up and running.

However, this method assumes a standard pattern for launching, debugging, and profiling an application. This assumption can be limiting for more advanced applications (for example, those with very complex `tile-monitor` command lines and/or profiling requirements).

It is recommended that you start with this method, and gradually migrate to invoking `tile-monitor` directly (for example, by using a Make Target or from a makefile) as your skills and needs grow.

5.11.1 Creating a Monitor Launch Configuration

To create a launch configuration for the sample application, do the following:

1. Select the `hello_world_pthread` project in the **C/C++ Projects** view, as shown in [Figure 5-10](#), “Hello World Project, After Build,” on page 46. (This makes the next step simpler, because the IDE automatically initializes a new launch configuration’s properties from the currently selected project, if any.)
2. In the **Monitor View**, right-click **Monitor Launch Configurations**, and select **Add Launch Configuration...**. See [Figure 5-13](#).

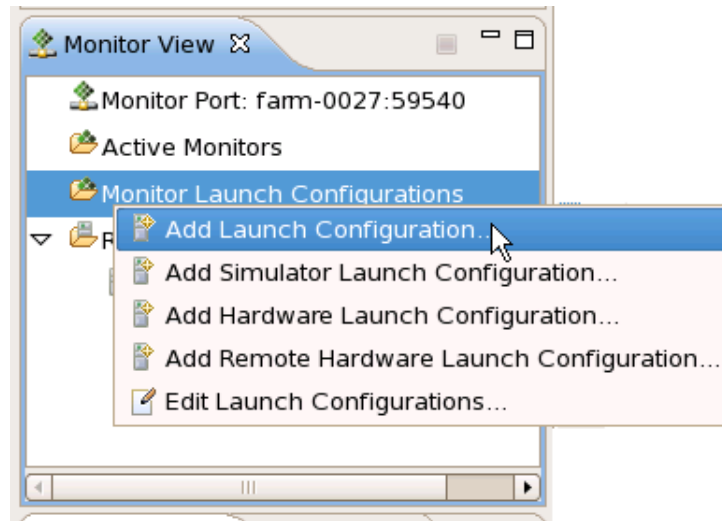


Figure 5-13: Add a Launch Configuration

This creates a new launch configuration and opens the **Launch Configuration Editor** to allow you to view and edit it. See [Figure 5-14](#).

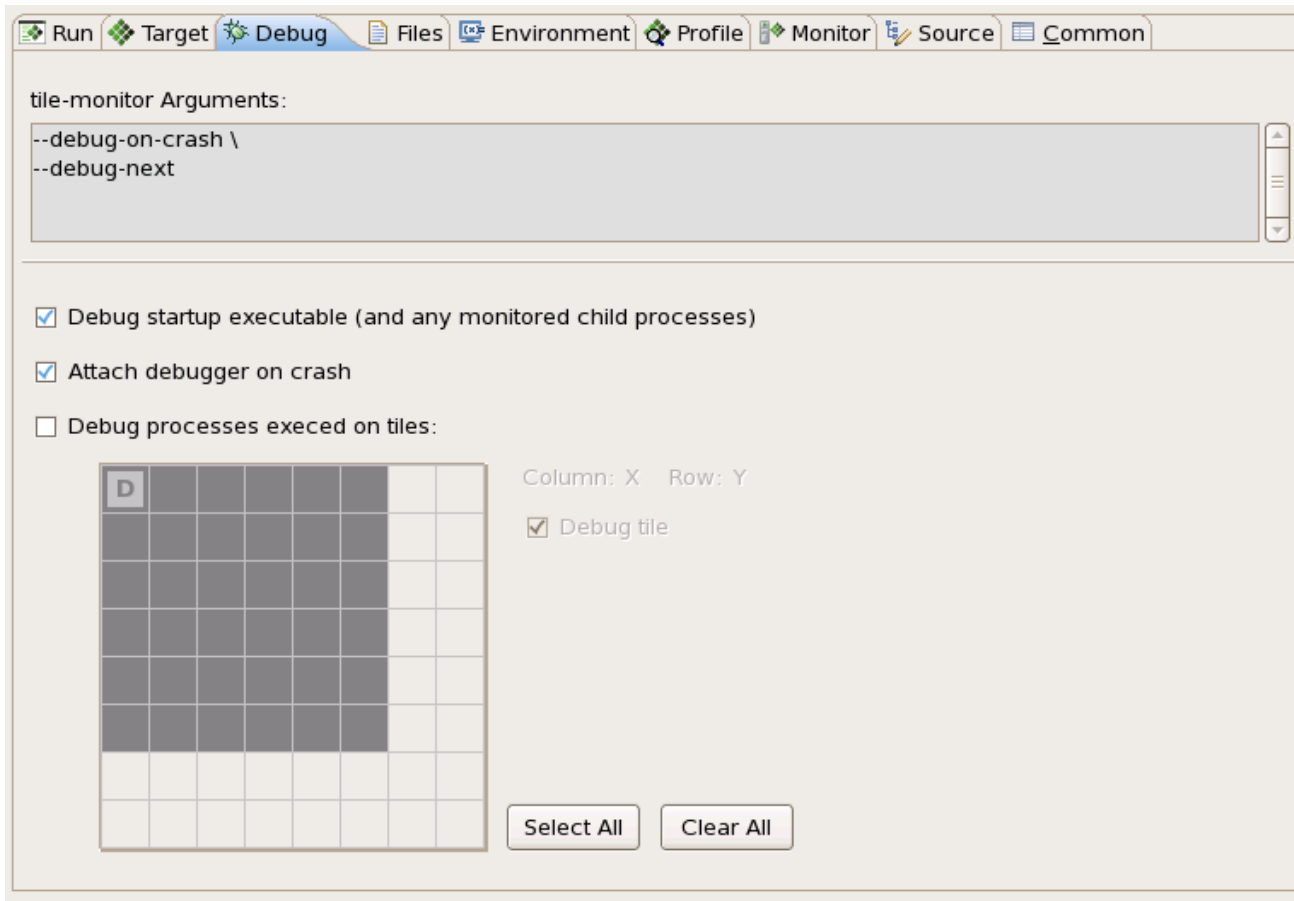


Figure 5-14: Launch Configuration Editor

The selected project's name has automatically been used as the name of the launch configuration. You can edit this, however, if, for example, the executable has a different name from the project directory.

The IDE has already filled in almost everything you need to run the application:

- The associated project
- The working directory to use in the Tile Processor's virtual file system
- The executable binary to run
- On the **Target** tab, an initial target selection
- On the **Debug** tab, initial debugging selections
- On the **Files** tab, a file-copy rule that copies the executable binary to the working directory
- On the **Monitor** tab, the initial tile-monitor location

Edit the target selection and tile-monitor location.

Note that at the top of the tab, the editor displays the implied tile-monitor command-line arguments based on the options selected on that tab. This is true for each tab in the editor, with the exception of the **Monitor** tab, which displays the complete implied tile-monitor command line that this launch configuration invokes.

Selecting a Hardware Target

We use the term *target* here to refer to the Tiler platform (hardware or simulator) on which the application is to be run.

Select a Hardware Target If Using a Tiler Device

In this example, you connect to a Tiler device running a TILE-Gx processor.

1. Select the **Target** tab in the **Launch Configuration Editor**.
2. Set the **Execution Target Type** to **Hardware**. Set the **Chip Model** and **Card Model** based on your hardware.

The **Target** tab looks like [Figure 5-15](#).

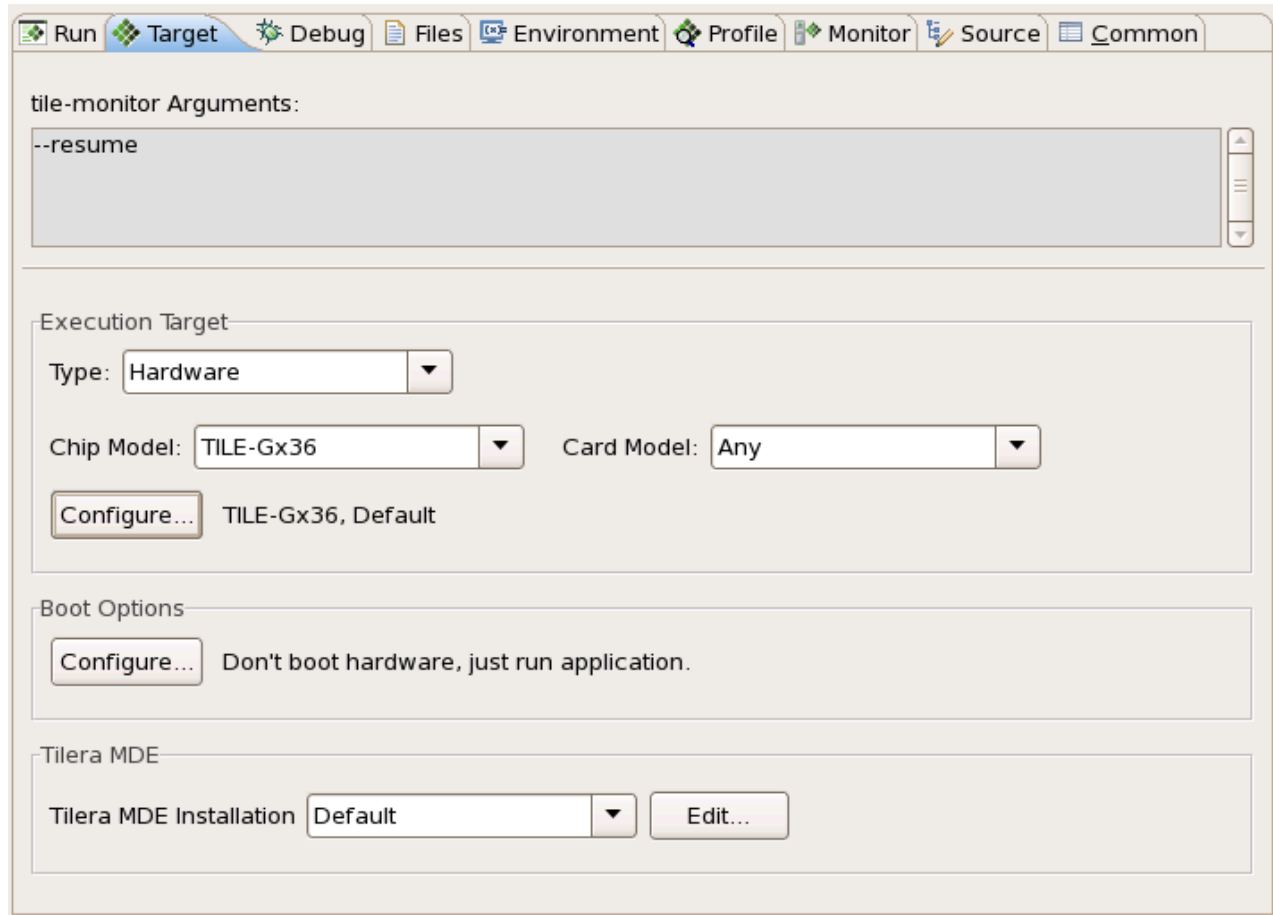


Figure 5-15: Target Tab, Hardware Target

3. In the **Boot Options** panel, click the **Configure...** button to display the **Boot Options** dialog.
4. In the **Boot Options** dialog, select **Boot Bootrom**, make sure the **Default Bootrom** check box is checked, and click OK to apply your changes. (See [Figure 5-16](#).) This tells tile-monitor to generate an appropriate default bootrom for the Tiler hardware and to use that bootrom to boot the hardware before running the application.

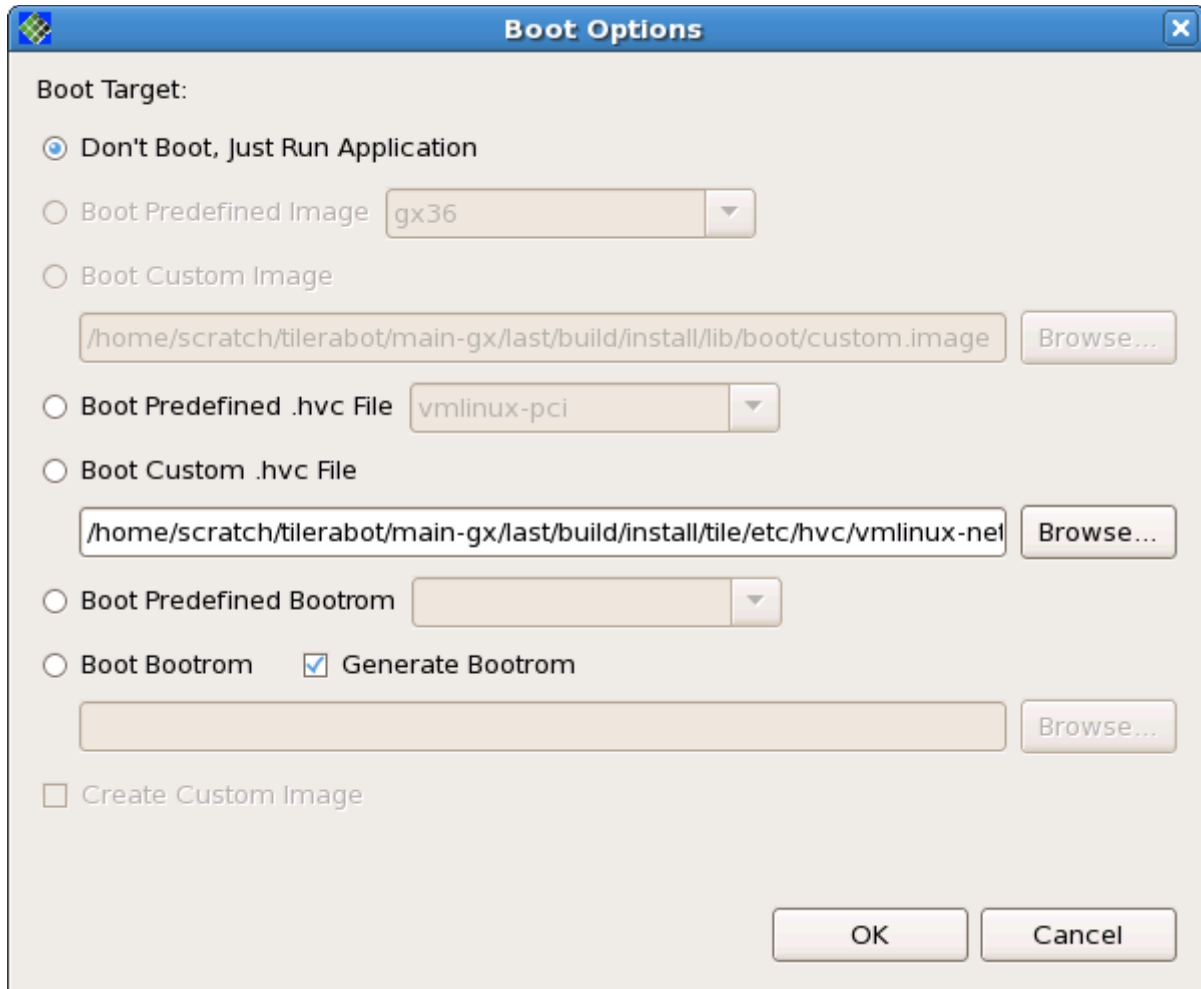


Figure 5-16: Target Tab, Boot Options Dialog

Select a Hardware Target If Using a Networked Device

If your target is a networked device, perform the following steps:

1. Set the **Chip Model** field as appropriate and leave the **Card Model** set to **Any**. Then click the **Configure...** button in the **Execution Target** panel to display the **Hardware Options** dialog.
2. On the **Hardware Options** dialog, set the **Hardware Type** to **Network Address**. Enter the device's network hostname or IP address in the **Address:** textbox. (For example: myhostname or 192.168.0.0.) Then click OK to apply your changes and close this dialog. See Figure 5-17.

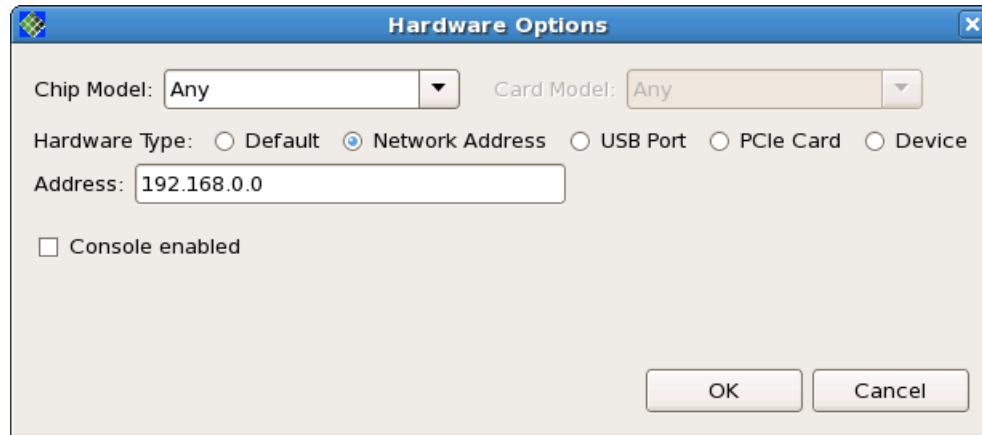


Figure 5-17: Hardware Options for a Networked Device Target

3. In the **Boot Options** panel, click the **Configure...** button to display the **Boot Options** dialog.
4. In the **Boot Options** dialog, make sure that the **Don't Boot, Just Run Application** radio button is turned on. This is necessary because the IDE cannot know how to boot networked devices. Ensure that the device is already booted before you attempt to run or debug applications on it.

Selecting the Monitor Host

Now you need to specify the *monitor host*, where the `tile-monitor` instance runs.

There are two options, which you can specify on the **Monitor** tab:

- **Local** — the Tilera hardware is installed in the host the IDE is currently running on
- **Remote** — the Tilera hardware is installed on some other remote host

We describe the Remote option here. Let us assume that you run the IDE on one host (for example, your desktop system), and the TILE-Gx device we want to run on is installed in a remote host. Configure the launch to connect to this host and run `tile-monitor` on it.

We also describe the “Remote Hosts” feature, which allows you to use the same launch configuration on multiple hosts.

Click on the **Monitor** tab, and do the following:

1. Under **Monitor Location**, select **Remote**.
2. If necessary, edit the connection command in the **Remote** textbox to be an appropriate command for connecting to the remote host and running a command on it. However, in place of the actual name of the remote host, use the variable `${remote_host}`. This tells the IDE we want to connect to a remote host, but we choose that host at run time. See [Figure 5-18](#).

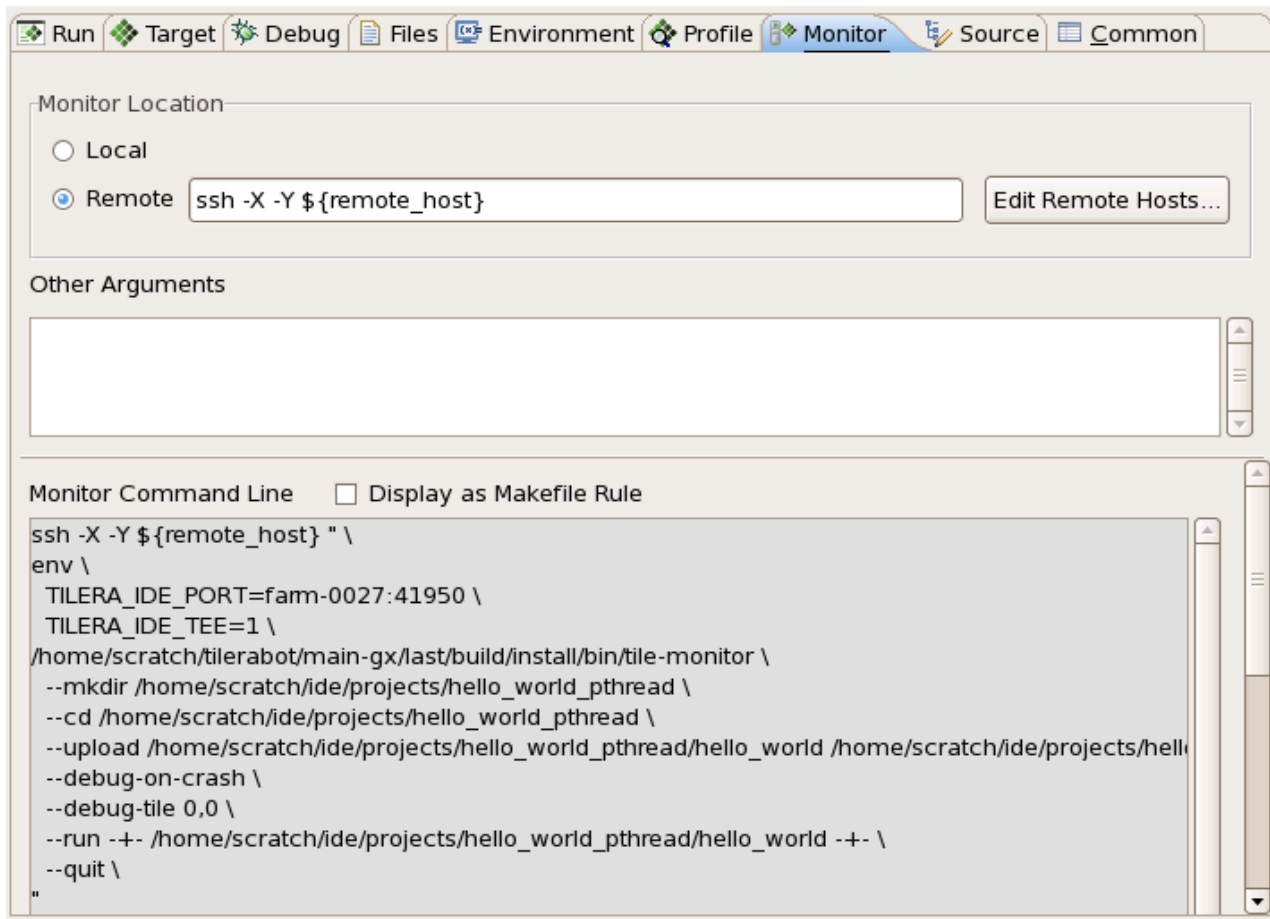


Figure 5-18: Monitor Tab

Now we need to add at least one remote host name:

1. Click the [Edit Remote Hosts...](#) button.
2. On the [Edit Remote Hosts](#) dialog, click [Add...](#) . See [Figure 5-19](#).

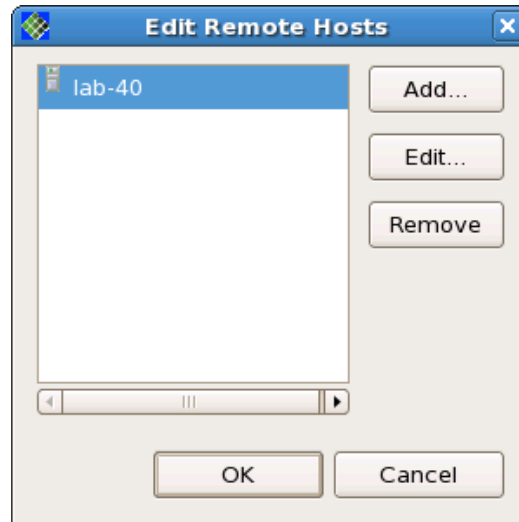


Figure 5-19: Remote Hosts Editor

3. On the **Add Remote Host** dialog, enter the name of the remote host.
4. Click OK on this dialog, and click OK to close the **Edit Remote Hosts** dialog as well. See [Figure 5-20](#).

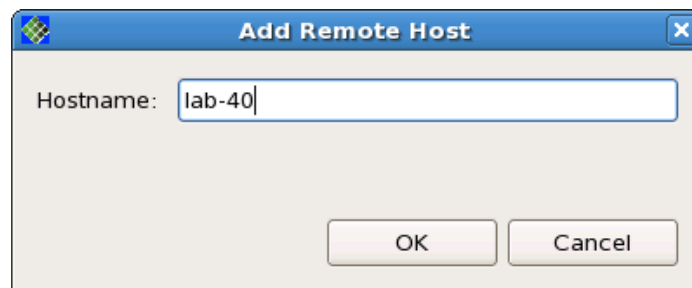


Figure 5-20: Add Remote Host Dialog

Note: If there is more than one remote host defined, when a launch configuration that uses the `${remote_host}` variable is invoked, the IDE displays the **Edit Remote Hosts** dialog again to let you select which remote host you want to run on. We have only defined one, so the IDE automatically uses that remote host each time.

Monitor Command Line

The **Monitor Command Line** textbox shows the entire `tile-monitor` command line that the IDE invokes when you launch this configuration. It should look something like [Figure 5-21](#).



Figure 5-21: Monitor Command Line (Sample)

Except for the reference to `${remote_host}`, you could copy and paste this command line to a shell to invoke `tile-monitor`. The command line includes the necessary environment variables to make `tile-monitor` connect back to the current instance of the IDE.

You could also copy this command line to a makefile rule. As a convenience, if you check the **Display as Makefile Rule** checkbox, the command line is displayed as a properly formatted makefile rule, ready to be copied and pasted. See Figure 5-22.

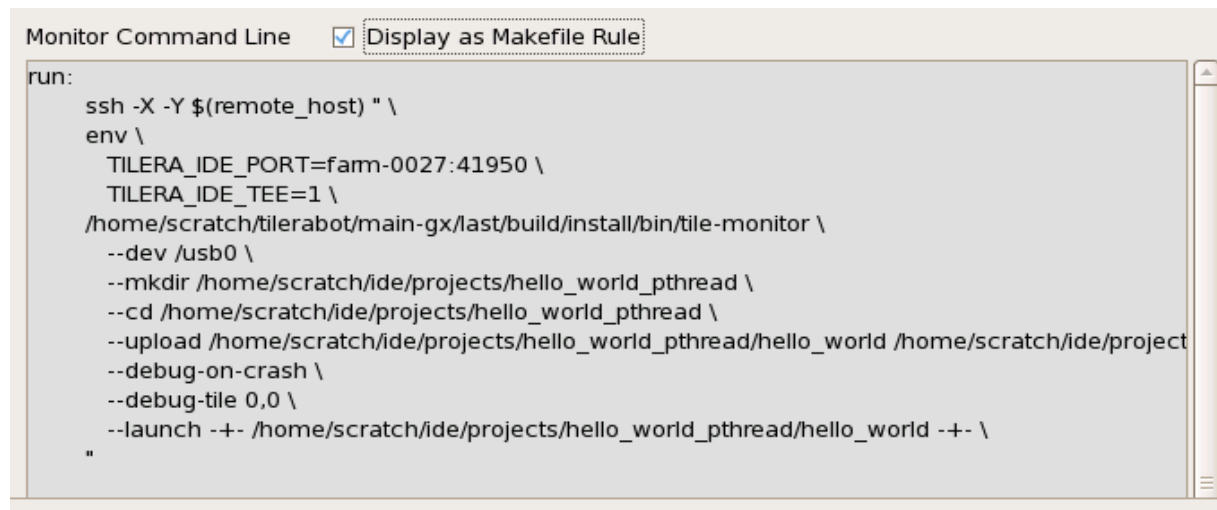


Figure 5-22: Monitor Command Line as Makefile Rule (Sample)

This generated makefile rule is intended as an example, and works only in the current IDE session. In particular, the IDE `hostname:port` information is based on the current IDE instance; it might be different the next time the IDE is run.

The [Tilera IDE Online Documentation](#) shows how to construct a makefile rule that is independent of the current IDE session. For more information, see the topic “Running Applications via Makefile” in the “IDE User’s Guide” section.

5.11.2 Closing the Launch Configuration

At this point, we have finished editing the launch configuration for the sample application. If necessary, click **Apply** to save any pending changes, and then click **Close** to close the **Launch Configuration Editor**.

5.11.3 Invoking the Launch Configuration

You can invoke the launch configuration directly from the editor. However, we are going to do it another way that is more convenient in your daily work.

In the **Monitor View**, right-click the `hello_world_pthread` launch configuration and select **Run**. See [Figure 5-23](#).

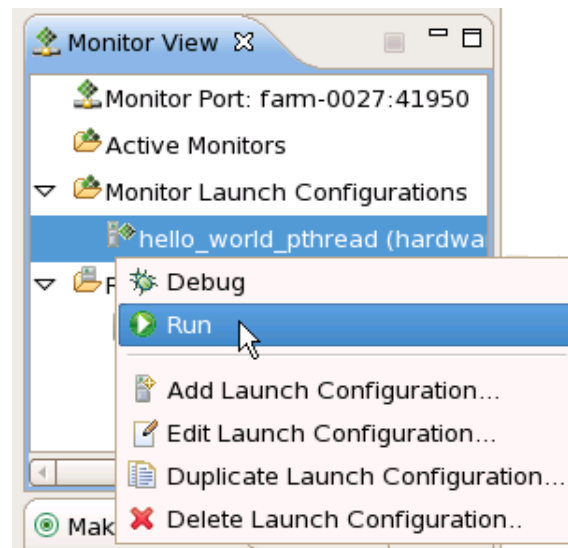


Figure 5-23: Running the Hello World Launch Configuration

This starts the sample application. However, the sample application completes so quickly that, aside from a small flicker of activity in one or more views, you might not see any output. By default, the launch configuration instructs `tile-monitor` to automatically shut down when it completes running the application. When `tile-monitor` shuts down, the IDE cleans up the connection and any console windows associated with that `tile-monitor` instance.

This is convenient if you have multiple `tile-monitor` instances connected to the IDE. It avoids clutter and is not normally an issue when you are debugging an application, because you usually stop at a breakpoint somewhere in the program before the program exits. This automatic cleanup can be an issue, however, if you simply want to run an application to view its output, as we are doing here.

Invoking `tile-monitor` Interactively

Modify the launch configuration so that `tile-monitor` runs in interactive mode and continues running after the application has exited, until you explicitly shut down `tile-monitor` yourself.

To do this, edit the launch configuration again. (In the **Monitor View**, right-click the `hello_world_pthread` launch configuration and select **Edit Launch Configuration...** from the menu. See [Figure 5-24](#).

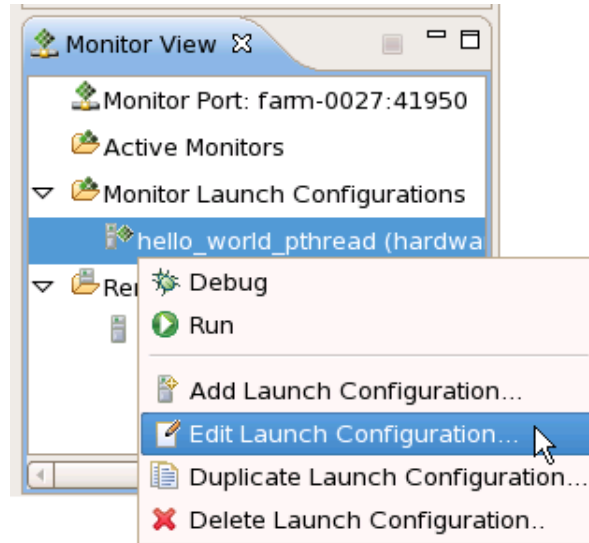


Figure 5-24: Editing the Hello World Launch Configuration

On the **Run** tab, check the box at the bottom of the tab labeled **Interactive Mode**. See Figure 5-25.



Figure 5-25: Run Tab's "Interactive Mode" Checkbox

This removes the `--quit` option from the end of the `tile-monitor` command line arguments, and also replaces the `--run` option used to invoke the application with `--launch`. This tells `tile-monitor` to launch your program and then immediately enter interactive mode while it is running, rather than waiting until the application has finished running. See Figure 5-26.

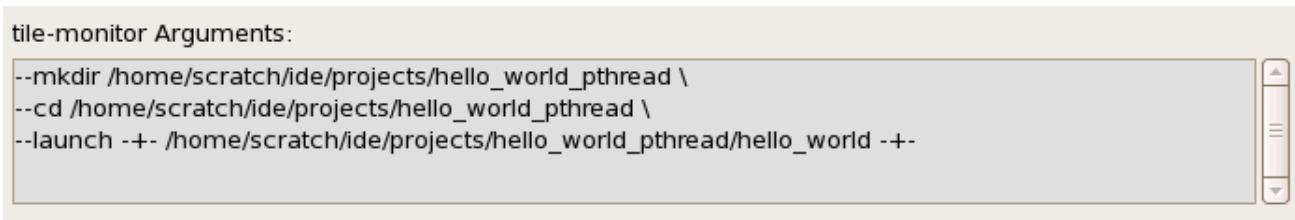


Figure 5-26: "Interactive Mode" Command Line Arguments

Apply the change, close the configuration editor, and run the configuration again. Now you should see output from the application, as shown in Figure 5-27.

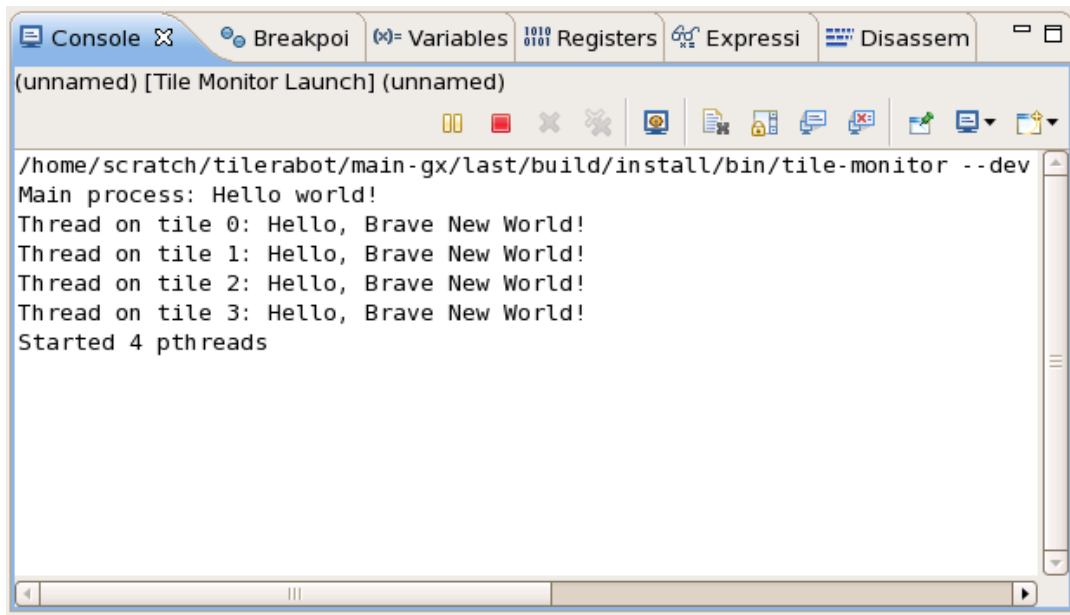


Figure 5-27: Output from Hello World Application

Shutting Down tile-monitor

At this point, the application has completed running, but because we are running `tile-monitor` in interactive mode, `tile-monitor` itself is still active.

The cleanest method to shut down `tile-monitor` is to type the `tile-monitor` command `\quit` into the **Console** view, as shown below.

Note: The `tile-monitor` command is actually `quit`, but you need to precede it with a `\` (slash) character when typing a command in the IDE's **Console** view, to distinguish it from standard input intended for a process.

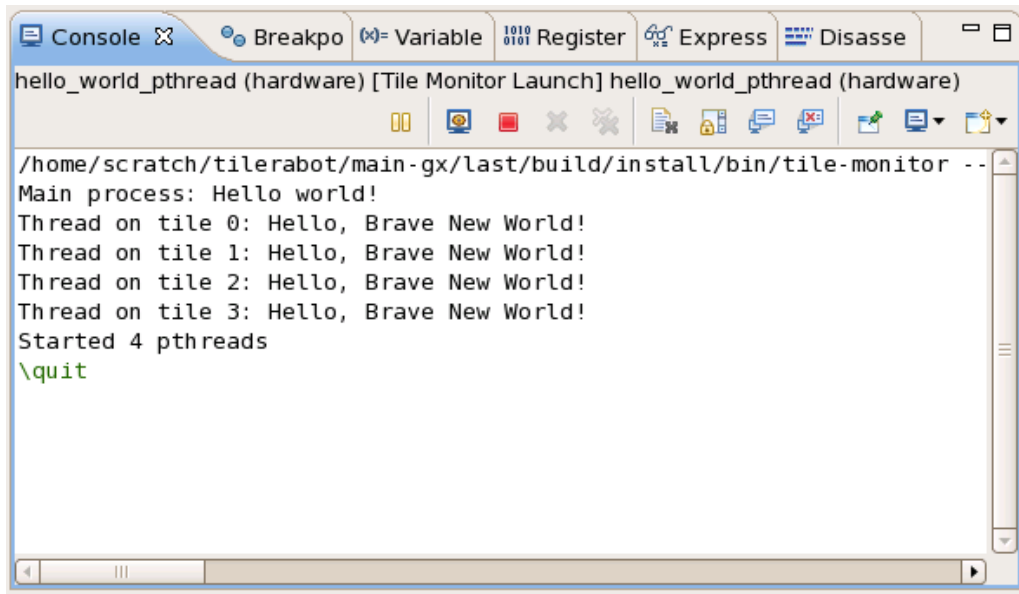


Figure 5-28: Using the quit Command to Shut Down tile-monitor

5.12 Continuing the Tutorial

This chapter has provided an introduction to the IDE and has virtually duplicated the information in these topics in the “Getting Started with the IDE” section in the [Tilerobot IDE Online Documentation](#):

- “The Sample Application”
- “Creating a Project”
- “Running the Application”
- “Running via Launch Configuration”

Now you can continue with “Running via Makefile” in the “Getting Started with the IDE” section in the [Tilerobot IDE Online Documentation](#).

In particular, the online documentation discusses using the IDE for debugging and profiling of applications. It also provides more detail on creating projects for applications and on running them, both from within the IDE and externally by means of makefiles.

CHAPTER 6 FREQUENTLY ASKED QUESTIONS (FAQs)

This chapter describes:

- [Section 6.1 Basic FAQs](#)
- [Section 6.2 FAQs About tile-monitor](#)
- [Section 6.3 FAQs About the IDE](#)
- [Section 6.4 FAQs About the Hypervisor](#)

6.1 Basic FAQs

How do I find out which version of the MDE I'm using?

Use this command: `tile-sim --fullversion`.

Where do I find information about developing applications in the Tilera environment?

See *Programming the TILE-Gx Processor* (UG505).

Where do I find information about optimizing my applications?

See the *Multicore Development Environment Optimization Guide* (UG515).

Where do I find information about my board?

For information about the board part number, serial number and revision, processor serial number, and so forth, see the files in the `/sys/hypervisor/board` directory, for example `board_description` and `switch_control`.

6.2 FAQs About tile-monitor

I got the error [monitor] ERROR: The file '/dev/tilegxpci0/lock' is currently locked! What does this mean?

This error can happen when another instance of `tile-monitor` is already using the TILE-Gx device, and you try to run a new instance of `tile-monitor`.

This indicates that another user is accessing the card. Find out who else is using the card. *Do not kill the process!*

6.3 FAQs About the IDE

Many of these FAQs can also be found in the [Tilera IDE Online Documentation](#).

When I try to start `tile-eclipse`, it says the workspace is already in use.

The `tile-eclipse` IDE is based on Eclipse, which has a guard mechanism to enforce the rule that, at most, one Eclipse instance can be running on a given workspace at a time.

First, make sure you do not already have an instance of `tile-eclipse` running on the same host. Remember that if you are running under X Windows, the application window might be displayed on a different screen. (For example, you are trying to run `tile-eclipse` from home, and have left an instance running at work.)

Next, check your process listing (for example, `ps -wx`) and look for any rogue `eclipse` or `java` instances. If Eclipse was not shut down cleanly, a windowless process might have been left behind. Kill any such processes. When the Eclipse process is shut down, it should release its lock on the workspace directory.

If the above does not work (or there are no Eclipse or Java processes currently running), manually delete the `.lock` file from your workspace directory. This should be located in:

```
${HOME}/.tilera/workspace/.metadata/.lock
```

You should now be able to start `tile-eclipse` normally.

Alternatively, use `tile-eclipse --force`, which deletes the `.lock` file and starts up `tile-eclipse` normally.

If you are using the `tile-monitor here` command to select the current directory as the workspace directory, remember that the `.lock` file, if there is one, exists in a subdirectory `.metadata` of the current directory.

I am unable to edit or rename files in the examples directory. What is wrong?

The MDE files might have been installed read-only, or be owned by `root`. The simplest way around this is to make your own copy of the `examples` directory in your home directory. This also allows you to edit and tweak the example without affecting other users.

How do I get additional diagnostics information?

The IDE generates log files in the workspace directory, recording information about its operation and its communication with other tools, such as `tile-monitor` and the `tile-gdb` debugger.

While these files do not contain much user-readable output, they are extremely helpful for Tilera developers who are responding to an IDE issue, since the logs give a very precise and detailed picture of what took place “behind the scenes”. It is strongly recommended that if you encounter an issue in the IDE, that you immediately export the current diagnostic log files, and forward the exported logs with your inquiry to Tilera technical support.

Note: The diagnostic log files do not contain any developer source code, nor any application data apart from any program output that was displayed in the **Console** View during execution.

You can capture the IDE diagnostic log files with the following menu command: **Help→Export Diagnostic Logs....** This command saves the exported logs in a `.zip` file, which you can include as an attachment to your e-mail to Tilera.

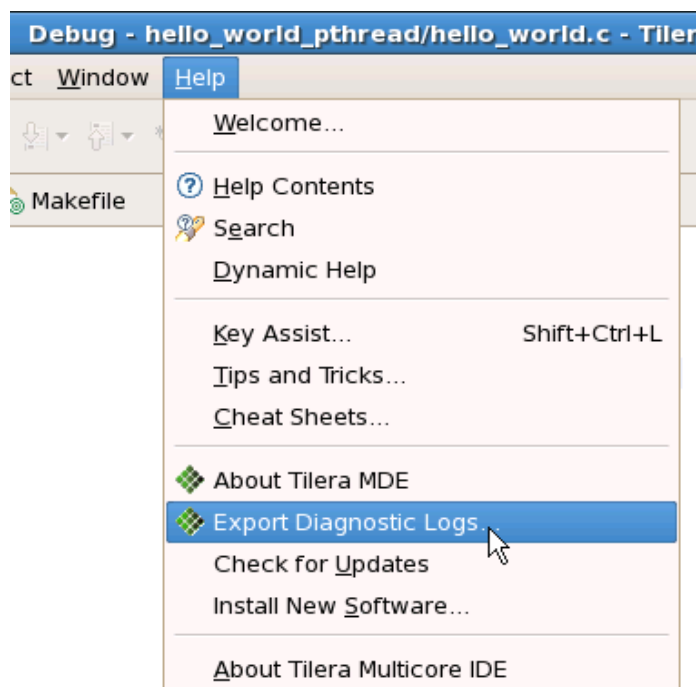


Figure 6-1: Export Diagnostic Logs Menu Command

If I delete a project from the workspace, can I get it back?

When you delete a project from the Eclipse workspace, if you answer “no” at the dialog that asks if you want to delete the actual project files, then the project content is still there, but simply not visible to Eclipse. You can re-import the project to make it visible again. To do this:

1. Select the menu command **File→Import**.
2. Select **General→Existing Projects into Workspace** on the **Import** dialog and click **Next**.
3. In the **Select root directory** option on the next tab, either type or browse to the old project directory.
4. Verify that the project’s name appears in the **Projects** list.
5. Click **Finish** to import the project.

I have closed one of the perspectives, so its name is no longer visible. How do I get it back?

1. Select the menu item **Window →Open Perspective →Other...**
2. Select the perspective name from the list, and click **OK**.

Note that there are two **Debug** perspectives; choose the one with the Tiler logo. The other is the generic Eclipse **Debug** perspective, which lacks Tiler-specific debugging views.

I have rearranged views in one of the perspectives. How do I restore them to the default positions?

Right-click on the perspective name in the top-right corner of the IDE window, and select **Reset** from the menu.

I have closed a view in one of the perspectives. How do I get it back?

1. Select the perspective by clicking its name in the top-right corner of the IDE window).
2. Select the menu item **Window → Show View...** and pick the view's name from the menu. If the view is not displayed on the menu, choose the **Other...** option and pick the view from the dialog.

How do I run a program that consists of multiple executables?

Specify the initial, startup executable on the **Run** Tab of the **Launch Configuration** editor. This executable should spawn instances of the others (through appropriate `exec()`/`fork()` calls, or by creating pthreads), and so forth).

If you are running on the hardware, remember that you need to use the **Files** tab to upload all the executables to the chip's virtual file system, or your program will fail when it attempts to spawn an instance of an executable that is not there.

Alternatively, you can create a makefile rule with a `tile-monitor` command line that specifies multiple executables. It is beyond the scope of this manual to enumerate all the possibilities. For more information, see the `tile-monitor` manual page in the Tileria Documentation Set.

Can I run an executable without having to create a project?

Yes. Follow these instructions:

1. Open the **Launch Configuration** editor, and double-click the **Tile Monitor Launch** configuration type to create a new launch configuration.
2. On the **Run** tab select the **No Project** radio button, and enter the working directory and application path for your executable.

Note: To be able to view source files when debugging your executable, you may need to also go to the **Source** tab and list the source directories for your program. If you are not seeing source files displayed when the debugger stops at a breakpoint, try adding the containing directory on the **Source** tab.

When I run a program, I get an error like: Error loading libcg.so: cannot restore segment prot after reloc: Permission denied. Why?

You are probably running MDE on Fedora core 6 (or higher). This error is likely because of the Security Enhanced feature of Linux (SELinux). There are two possible solutions to try:

- Change the security permissions for the shared library. For example:

```
$ cd $TILERIA_ROOT/lib/tile-cc
$ chcon -t texrel_shlib_t libcg.so
```
- Disable SELinux altogether (check with your system administrator to make sure this is acceptable). Edit the file `/etc/sysconfig/selinux` and set this value:
`SELINUX=disabled`

The IDE seems to get stuck when I run a large profile file. How can I run a large profile file?

If you want to load a very large profile, you must use `tile-eclipse --max-memory N` to pro-actively allocate more memory at IDE startup. *N* represents the maximum heap size to be used by the IDE, with a suffix indicating the size unit: `m` or `M` for megabytes, and `g` or `G` for gigabytes.

The default is 512M (512 megabytes).

For example:

```
$ tile-eclipse --max-memory 2G --profile profile_file.xml
```

6.4 FAQs About the Hypervisor

What do I do if I want to tweak the existing .hvc or .hvh files for my application?

There are many variables in the standard .hvc and .hvh files. You can see these files in `$TILERA_ROOT/tile/etc/hvc/` (or `/etc/hvc/` on a native TILE installation).

For example, the default memprof driver, which provides information for `/proc/tile/memprof`, consumes an entire tile. If you do not plan to use the information in `/proc/tile/memprof`, you can suppress the driver by adding `--hvd MEMPROF=0` to your `tile-monitor` command line.

If you want to change the name of the client, the tiles it runs on, the hash-for-home tiles it uses, the memory allocated to it, you can create your own .hvc file that includes one of the standard .hvc files, but first requests that the standard file skip its own client definitions, instead specifying all the client information.

For example, to create a Linux client that only uses a 4x4 grid, create the file `4x4.hvc`. The first line requests that the standard file skip its own definition of the client, and the second line includes a standard .hvc file:

```
define VMLINUX 0
include vmlinux.hvc
client vmlinux 4x4
  args TLR_NETWORK=auto $XARGS
```

To boot using this custom .hvc file, run `tile-monitor --hvc 4x4.hvc`.

To create your own custom hypervisor configuration, run `tile-monitor` with the `--create-hvc custom.hvc` argument. This creates a file named `custom.hvc` that you can edit however you like. Feed this modified file back to `tile-monitor` using `--hvc custom.hvc`.

INDEX

Symbols

"hello world" application [40](#)

A

application
 defined [40](#)
architecture documentation [17](#)
audience of this guide [vii](#)
Author perspective
 definition of [39](#)

B

backups
 performing [10](#)
browser
 using [35](#)
burning a recovery image [9](#)

C

changes from previous release [vii](#)
configuring SELinux [14](#)
context-sensitive help [36](#)
conventions
 notation [ix](#)
creating a download directory [10](#)
customer support [viii](#)

D

Debug perspective
 definition of [39](#)
development process steps [22](#)
development tools documentation [18](#)
diagnostics information [62](#)
document organization [vii](#)
documentation
 development tools [18](#)
 I/O device [18](#)
 libraries [18](#)
 operating system internal interfaces [18](#)
 target platform tools [18](#)
 Tile Processor architecture and hardware [17](#)
download directory
 creating [10](#)

E

Eclipse [37](#)
 Tilera extensions to [39](#)
environment variable TILERA_WORKSPACE [38](#)
environment variables [4](#)
example application
 using [41](#)
example program
 multi-tile [29](#)
 running on network-attached system [28](#)
 single-tile [28](#)
examples directory [41](#)
 editing and renaming files [62](#)
external project
 definition of [38](#)

F

file
 restoring [11](#)
filesystem
 rebuilding [10](#)
--functional [22](#)

G

gen numbers
 promoting [9](#)
getting help
 on the IDE [34](#)
getting started [33](#)

H

hardware
 documentation [17](#)
 platform, defined [22](#)
help
 on MDE commands [27](#)
hvconfig file [12](#)
hypervisor [12](#)

I

I/O device documentation [18](#)
IDE
 getting help [34](#)
IDE online help [viii](#)
info pages [viii](#)

internal project
definition of 38

L

large profile file 64
libraries documentation 18

M

man pages viii
MDE
components
overview 21
document Index viii
document index
location of viii
documentation
location of viii
version 61
mounting partitions 10
multiple executables 64

N

New Project Wizard 42
closing 44
notation conventions ix

O

online help 34, 36
online help browser
using 35
operating system internal interfaces documentation 18
organization of this guide vii

P

partitions
mounting 10
PATH environment variable 4
performing
backups 10
perspective
closing 63
defined 39
definition of 39
Profile perspective
definition of 39
program
defined 40
project
building 44
creating 41
defined 39
definition of 39
deleting from workspace 63
directory
definition of 38

project directory
definition of 38
promoting the gen numbers 9

R

rebooting the platform 11
rebuilding the filesystem 10
recovery image 9
burning 9
restoring files 11
running
executable 64

S

SELinux
configuring 14
simulator
platform, defined 22
SPI-ROM
updating 11
steps in the development process 22
support
technical or customer viii

T

target platform tools documentation 18
technical support viii
Tile Processor documentation 17
tile-env
using to set your environment variables 4
Tilera extensions to Eclipse 39
TILERA_ROOT environment variable 4
TILERA_WORKSPACE environment variable 38
TILExtreme-Gx36 and TILExtreme-Gx36 Duo platform
rebooting 11
tutorial 33
using 40

U

updating the SPI-ROM 11
using the online help browser 35

V

view
closing 64
defined 38
definition of 38
rearranging 63
restoring 63

W

workspace directory 37
definition of 37